

12. 算法部分展开讲：从回波到消光、PM 和热点

这一节开始，我们不再停留在“知道有哪些算法”，而是逐步讲清楚数据是怎么一步一步被变成业务指标的。

12.0 先把 LiDAR 方程看成 4 个功能块

 LiDAR 方程分块图

如果你一看到公式就紧张，可以先完全不碰推导，只把它当成一条“回波是怎么被制造出来的因果链”。

单波长弹性 LiDAR 的核心方程可以先粗略读成：

测到的回波 = 仪器刻度 $\times$ 距离和重叠修正 $\times$ 这一层打回来多少 $\times$ 来回路上损耗

这张图真正想表达的是：

1. 回波强弱不只取决于颗粒物本身。
2. 仪器强不强、远近距离造成的几何扩散、近场重叠是否充分、这一层会不会把光打回来、来回路上被空气吃掉多少，都会同时影响结果。
3. 真正和颗粒物关系最密的是 $\beta(R)$ ，也就是“这一层把多少光打回 LiDAR”。
4. 但 $\alpha(R)$ 也很关键，因为它决定光在去程和回程中被削弱多少。
5. 反演之所以难，是因为你只测到了一个回波 $P(R)$ ，但里面至少混着后向散射 $\beta(R)$ 和消光 $\alpha(R)$ 这两个未知量。

所以对数学基础弱的读者来说，最重要的不是先背公式，而是先记住这个因果关系：

曲线变高，不一定只是颗粒物变多；曲线变低，也不一定就是颗粒物变少。它可能是仪器刻度、距离几何、近场重叠、后向散射或传播衰减共同作用的结果。

工程处理的顺序也基本沿着这条因果链走：

1. 先做仪器和背景相关的预处理，让 $P(R)$ 尽量可信。
2. 再做距离平方校正和重叠校正，把明显的几何影响拿掉。
3. 然后才用 Klett/Fernald、参考区段和消光-散射比例等假设，把 $\beta(R)$ 和 $\alpha(R)$ 从同一条回波曲线里尽量分开。

12.1 第一个能看的图层：衰减后的后向散射

这一节原来叫“衰减后向散射”，英文是 **attenuated backscatter**。英文名很吓人，但它的意思可以先翻译成：

一层空气把光打回来了多少，不过这个结果还带着“路上被削弱”的影响。

也就是说，它不是最终 PM2.5，也不是完全真实的颗粒物浓度。它更像一张“哪里有结构、哪里可能有污染团”的初步图层。

为什么远处天然更暗——其实是两件事叠加

原始回波 $P(R)$ 随距离变弱，原因有两个，必须分开看：

$$P(R) = C \cdot \frac{\beta(R)}{R^2} \cdot T(R), \quad T(R) = \exp\left(-2 \int_0^R \alpha(r) dr\right)$$

1. **几何扩散** $1/R^2$ ：光是一路发散的球面波，越远单位面积上的光越稀。这一项纯粹是“距离造成的变暗”，和空气脏不脏无关，**乘上 R^2 就能抵消**。
2. **双程透过** $T(R)$ ：LiDAR 不是单向照射，而是“打出去 → 颗粒物散射回来 → 再走回雷达”的**双程**过程。去程被消光吃一次、回程再吃一次，所以定义里指数上直接带 -2 ——这个 2 就是“一来一回走两遍”的体现。只要空气里有消光 ($\alpha > 0$)， $T(R)$ 就**单调下降**，而且**乘 R^2 抵消不掉**。

把 R^2 乘掉之后，得到的就叫**衰减后向散射**（差一个仪器常数 C ）：

$$X(R) = P(R) \cdot R^2 = C \cdot \beta(R) \cdot T(R)$$

关键结论： R^2 校正只拿掉了“几何变暗”，“传播衰减” $T(R)$ 还留在里面。所以**即使空气全程一样干净、一样均匀， $X(R)$ 也会随距离缓慢单调下降**——绝不是被 R^2 校正“拉平”。

那回波会在遇见污染团时增大吗？——会，而且这正是我们要找的信号

会。原因是 $X(R) = C \cdot \beta(R) \cdot T(R)$ 里有两个相反的趋势：

- $T(R)$ 永远在下降（光一路被吃）；
- $\beta(R)$ 在污染团里会**猛增**（颗粒多 → 把光打回来的能力强）。

当污染团里 β 的增量**超过** $T(R)$ 的下降量时， $X(R)$ 就会出现一个**局部凸起 (bump)**。这就是“回波在污染团处反而变大”的真正机制——不是校正造成的假象，而是颗粒物后向散射的真实增强。

更有意思的是凸起之后的样子：穿过污染团后 β 回落，而团内很高的消光已经**额外蚕食了** $T(R)$ ，所以 $X(R)$ 在团后会**比来时降得更陡**。“先凸起、后骤降”就是污染团在衰减后向散射图上的经典签名。

先看示意图：



一个物理自治的假数据例子

下表用同一套 β 、 T 算出 $X = \beta \cdot T$ （为好读， β 、 X 用任意单位，常数 C 取 1）。在 2.5 km 处放一团污染（ β 涨到 4 倍）：

距离 R	后向散射 β	双程透过 T	衰减后向散射 $X = \beta T$	怎么读
0.5 km	1.0	0.90	0.90	近端，干净
1.5 km	1.0	0.72	0.72	仍干净， X 随 T 缓降
2.5 km	4.0	0.50	2.00	污染团： $\beta \times 4$ ， X 反而凸起
3.5 km	1.5	0.20	0.30	刚过团： β 回落+ T 被吃，骤降
4.5 km	1.0	0.10	0.10	远端，信号已经很弱

对照实验：如果全程没有污染团 ($\beta \equiv 1$)， X 会是 $0.90 \rightarrow 0.72 \rightarrow 0.58 \rightarrow 0.47 \rightarrow 0.38$ ，一路单调缓降。拿它和上表比就能看出——2.5 km 那个凸起**不是 R^2 校正造成的**，而是污染团后向散射真实增强压过了传播衰减。

再强调一次方向：原始回波 $P(R) = X/R^2$ 因为还带着 $1/R^2$ ，下降得极陡，污染团的凸起几乎被“几何变暗”埋没；做 R^2 校正得到 $X(R)$ 后，凸起才露出来。但 $X(R)$ 里仍留着 T ，所以它只能用来“先看形状、找污染团位置”，还不能直接当浓度。

这张图层的用途是“先看形状”，不是“直接报 PM”。你可以把它理解成医生先看 X 光片：能看到异常位置，但还不能只靠一张片子就给出全部诊断。

12.2 参考区段为什么重要

Klett/Fernald 这类算法不是魔法。它想从一条回波曲线里反推出整条路径上的空气状态，但它需要先找一个“可信起点”。

这个可信起点通常放在远端，叫参考区段。英文里常写成 **reference range**，符号常写成 R_c 。你第一遍只记中文“参考区段”就够了。

更通俗地说：

先找一段相对干净、相对稳定的空气，当成尺子的起点，然后算法再从这里往回推。

参考区段逻辑图

选参考区段时，最怕两种情况：

1. 选到污染团里：算法会把“脏空气”误当成干净起点。
2. 选到噪声很大的远端：算法从一开始就不稳，后面整条结果都会漂。

一个假数据例子：

距离段	校正后信号	质量判断	适合当参考区段吗
200-300 m	180, 220, 260	变化大，像污染团	不适合
500-600 m	130, 128, 132	稳定，信号还够	适合
900-1000 m	25, 18, 30	太弱，噪声大	不适合

所以“参考区段”不是随便选一个远处位置，而是在问：

哪一段空气最像一个可靠的起跑线？

12.3 Klett/Fernald 到底在算什么

先把英文名放一边。你可以把 Klett/Fernald 理解成一种“带起点的反推算法”。

LiDAR 只测到一个东西：回波曲线。

但回波曲线里混着两件事：

- 1. 这一层空气把多少光打回来了，记作 β 。
- 2. 光在路上被吃掉了多少，记作 α 。

这两个量的中文直觉是：

符号	中文叫法	你可以怎么想
β	后向散射	这一层把多少光“打回雷达”
α	消光	光经过这里时被“吃掉/削弱”多少

问题是：雷达只看到“最后回来的亮不亮”，却不知道是因为“这一层打回来得多”，还是因为“路上被吃掉得少”。

所以算法需要再加一个经验假设：消光-散射比例。英文常写成 `lidar ratio`。

消光-散射比例。

它常写成：

$$S = \frac{\alpha}{\beta}$$

不用急着推公式。先按这句话理解：

S 是在说：同一类颗粒物，通常“吃掉光的能力”和“把光打回来的能力”大概是什么比例。

一个假数据例子：

场景	后向散射 β	假设 S	推出来的消光 α	直觉
干净空气	0.001	50	0.05	吃光少
普通扬尘	0.004	50	0.20	吃光明显
浓烟/浓尘	0.010	50	0.50	吃光很强

这张表不是要你记数值，而是要你看懂关系：

先从回波里估“打回来多少”，再用比例关系推“路上吃掉多少”。

12.4 Klett 反演怎么理解：不是背公式，而是做反推题

先回答一个很容易卡住的问题：

Klett 不是英文缩写，而是人名。

Klett 反演指的是美国学者 James D. Klett 提出的一类弹性 LiDAR 反演方法。你可以先把它理解成“用 Klett 这个人提出的方法，从回波反推出消光和后向散射”。

Klett 的论文公式会写成积分形式，看上去很复杂。入门阶段先不要被积分吓住。它在程序里做的事情可以翻译成一句很土但很有用的话：

先在远处找一个比较可信的参考点，然后从远处往近处，一格一格把“这层空气有多浑浊”算回来。

下面用一组假数据完整走一遍。

12.4.1 假设我们已经拿到一条回波

为了让数字简单，我们只取 5 个距离点：

距离 R	原始回波 P	先不要急着解释
100 m	300	近处回波通常很大，因为距离近
200 m	60	继续变小
300 m	32	这里下降变慢了，可能有污染团
400 m	13	猛地掉下去
500 m	6	远端参考点

注意原始回波 P 是一路单调下降的，哪怕 300 m 那里可能有污染团， P 也照样在掉——因为 $1/R^2$ 几何变暗太强，污染团的增强几乎被埋没。这正好对应 12.1 节强调的事实：**原始 P 不能直接读成污染浓度。**

第一步先做距离平方校正。这里为了计算方便，把距离写成 km：

$$X(R) = P(R) \times R^2$$

这里的 $X(R)$ 可以先理解成“距离平方校正后的回波”，也就是前面讲过的 RCS。

距离 R	R(km)	原始回波 P	R^2	$X = P \times R^2$
100 m	0.1	300	0.01	3.00
200 m	0.2	60	0.04	2.40
300 m	0.3	32	0.09	2.88
400 m	0.4	13	0.16	2.08
500 m	0.5	6	0.25	1.50

注意这个变化（重点！）：

X 整体仍在下降（3.00→2.40→2.88→2.08→1.50，远端比近端低），
但 300 m 出现了一个局部凸起——它既比 200 m 高（2.88 > 2.40），也比 400 m 高（2.88 > 2.08），
但并没有超过 100 m 的 3.00。

这个凸起才是 300 m 附近真正有污染增强的信号，而不是“乘 R^2 把远处拉高了”。
乘 R^2 只是抵消了几何变暗，传播衰减 $T(R)$ 还在，所以 X 的大趋势仍然是越远越低，100 m 始终最高。

12.4.2 给 Klett 一个远端参考点

Klett 不能凭空开始算。它需要一个参考点。

这里我们假设：

参数	数值	通俗解释
消光-散射比例 S	50 sr	把“打回来多少”换算成“吃掉多少”的比例
参考距离	500 m	最远端这格看起来比较平稳
参考后向散射 β_{ref}	0.0035	先告诉算法：500 m 这一格大概是这个浑浊程度

因为：

$$\alpha = S \times \beta$$

所以参考点的消光大约是：

$$\alpha_{\text{ref}} = 50 \times 0.0035 = 0.175 \text{ km}^{-1}$$

这一步的直觉是：

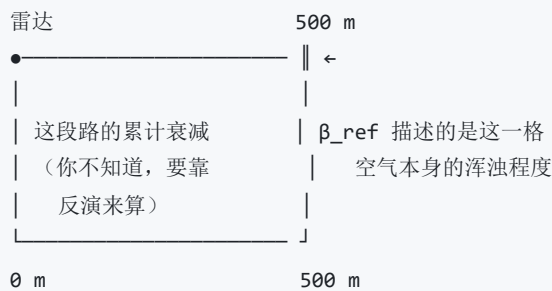
我先告诉算法“500 m 那一格大概有多浑浊”，然后让它从 500 m 往 100 m 反推回来。

🌀 β_{ref} 是从哪来的？能不能直接用干净空气的后向散射系数？

你的理解方向是对的，但有两个细节需要厘清。

第一： β_{ref} 是“这一格空气本身”的浑浊程度，不是“从雷达到这里一共衰减了多少”

这是最容易搞混的地方。 $\beta_{\text{ref}} = 0.0035$ 描述的是 **500 m 处那一小格空气** 把光打回来的能力。它是一个 **局地量** (local)，就像你说“北京今天 PM2.5 是 80”——说的是此刻、此地、这一格的浓度，不是从上海到北京的累计。



所以 $\alpha_{\text{ref}} = S \times \beta_{\text{ref}} = 0.175 \text{ km}^{-1}$ 的意思是：“**500 m 处空气本身的消光能力大约是每公里衰减 17.5%**”，而不是“光从 0 走到 500 m 总共被削弱了 17.5%”。

第二： β_{ref} 能不能直接拿干净空气的值来用？——能，但这是“假设”，不是“测量”

你的直觉是：如果 500 m 处空气很干净，那 β_{ref} 就可以用纯分子大气 (Rayleigh 散射) 的理论值代入。**这个做法在实际中确实经常这么做**，叫做“取远端清洁参考点”。具体操作是：

1. 查大气标准模型或用实测气压/温度，算出该高度的纯分子后向散射 β_{mol}
2. 假设 $\beta_{\text{ref}} \approx \beta_{\text{mol}}$ (认为这里没有气溶胶)
3. 用 $\alpha_{\text{ref}} = S \times \beta_{\text{ref}}$ 算出参考点消光

但要注意——**这只是一个假设，不是设备测出来的**。设备测到的是回波 $P(R)$ ，不是 β 。你把 β_{ref} 设成多少，会直接影响整条反演结果的绝对值：

β_{ref} 的取值	含义	对结果的影响
取偏小（假设很干净）	认为 500 m 几乎没气溶胶	整条廓线的 β 、 α 会整体偏小
取偏大（假设有气溶胶）	认为 500 m 还有不少颗粒物	整条廓线会整体偏大
取对了	参考点确实接近你假设的状态	结果最接近真实

这就是 Klett 的一个固有局限：**绝对值取决于你对参考点的假设**。不过好消息是——参考点假设对近场结果的影响远小于对远场的影响，因为你越往近处推，路径越短，假设偏差的权重越小。

第三：那"激光到 500 m 的累计消光"是什么时候算的？

累计消光不是在参考点这一步直接算出来的，而是在后面的 Klett 递推过程中**自动得到的副产品**。具体来说：

- 你先锚定 500 m 的 β_{ref} （已知）
- 往回推到 400 m，得到 $\beta(400\text{m})$
- 再推到 300 m，得到 $\beta(300\text{m})$
-一路推到 100 m

每推一格，你既得到了这一格的 β ，也顺便知道了"从这一格到 500 m 的累计消光"（就是公式分母里的积分项）。等你推到 100 m 时，0 到 500 m 的完整消光廓线就全部算完了。

一句话总结： β_{ref} 是一个你人为设定的**局地假设值**（可以拿干净空气的理论值来设，也可以用经验值），它不是设备测的，也不是累计消光；真正的"从雷达到 500 m 一共衰减了多少"是反演跑完之后自然得到的结果。

12.4.3 程序里真正用到的简化 Klett 计算

下面这个式子看着像公式，其实只对应代码里的三件事：

$$\beta_i = \frac{X_i}{X_{\text{ref}}/\beta_{\text{ref}} + 2S \int_{R_i}^{R_{\text{ref}}} X(r) dr}$$

如果你暂时不想看公式，就把它翻译成：

这一格的后向散射 =
 这一格校正后的回波
÷
 （参考点提供的起点 + 从这一格到参考点之间累计的衰减影响）

其中最关键的是这个"累计的衰减影响"。程序不会真的手算积分，而是把每两个距离点之间的面积加起来。

 **为什么 Klett 不从近处往远处算，而是从远处往近处算？**

这是入门时几乎所有人都会问的问题——“我先算近的，再一步步推到远的，不是更自然吗？”直觉上确实如此，但 Klett 偏偏要反着来。下面用三个生活类比说清楚。

类比 1：隔着雾看远处的灯

想象你站在一条大雾弥漫的走廊里，走廊上每隔 50 m 挂一盏灯。你想知道每盏灯本身有多亮（这就是后向散射 β ）。

问题来了：你能看到 300 m 处那盏灯，但你看到的亮度**不是它本身的亮度**——中间 0 到 300 m 的雾已经把它挡暗了一大截。而且你不知道中间的雾到底有多浓，因为雾的浓度正是你想算的东西。

`text` 你 300 m 处的灯 ●——雾？雾？雾？雾？雾？——💡 ↑ 这段路的雾有多浓？——你不知道！ 但正是这段雾决定了你看到的灯有多暗

如果从近处开始算，你首先得知道“0 到 50 m 的雾有多浓”，才能算 50 到 100 m。但 0 到 50 m 的雾浓度——你又得靠更近的灯来推……可最近的灯也在 50 m，它的亮度又被 0 到 50 m 的雾挡过了。**死循环**。

反过来，从远端开始：你在 500 m 处选一盏灯，假设它本身的亮度大概是多少（参考点），然后往回推——“如果 500 m 的灯是这个亮度，那从 400 m 看到 500 m 之间这段雾有多浓？”每一步只需要用到**已经算过的外侧路径**，不需要猜内侧未知的东西。

类比 2：传话游戏（误差为什么会被放大）

你玩过传话游戏吗？10 个人排成一排，第 1 个人悄悄跟第 2 个人说一句话，第 2 个人再跟第 3 个人说……到最后一个人听到的往往已经面目全非。

````text`

从近往远算（误差放大）：

100m → 200m → 300m → 400m → 500m

偏差 偏差<sup>2</sup> 偏差<sup>3</sup> 偏差<sup>4</sup> 完全失控！

↑

只要有一点误差，越传越离谱

`````

在 Klett 公式里，消光系数 α 出现在**指数**里面（不是普通的加减乘除）。这意味着近处的误差不是“线性增长”，而是“指数增长”——就像银行复利，利滚利。实际测试表明，如果 100 m 处的消光系数偏了 5%，传到 2 km 处偏差可能超过 100%，结果完全不可用。

````text`

从远往近算（误差收敛）：

500m ← 400m ← 300m ← 200m ← 100m

参考点 每一步只用已算过的外侧路径

(已知) 误差不会向内传播，越算越稳  
...

从远端参考点往回算时，每一步用到的积分是从**当前点往外到参考点**的累计，这段路径上的回波数据你已经全部拿到了（不需要猜），所以每一步都建立在已知数据上，误差不会像滚雪球一样越滚越大。

类比 3：从终点倒着走迷宫

如果让你从迷宫入口出发走到出口，中途有很多岔路，你不知道哪条对。但如果有人告诉你出口在哪里，然后让你**从出口倒着走回来**，每一步只需要确认"上一步是从哪个方向来的"，就容易多了。

Klett 的远端参考点就是那个"已知的出口"——你先假设远处某一点的浑浊程度（比如"500 m 处基本上是干净空气"），然后从这个已知点出发，一步步往回算每一层到底有多浑浊。

总结成一张表：

|       | 从近往远算 ❌               | 从远往近算 ✅           |
|-------|-----------------------|-------------------|
| 起点条件  | 近处信号被前方未知衰减"污染"，无法当起点 | 远端选一个干净参考点，条件已知   |
| 误差传播  | 指数放大（复利效应），越远越离谱      | 向内不传播，每步只用已算路径    |
| 结果稳定性 | 极不稳定，几乎不可用            | 稳定，Klett 证明了该方法收敛 |
| 一句话   | 死循环：要算近处就得先知道更近处      | 锚定远端，逐步回收         |

**一句话总结：**从近往远算，你需要先知道"前面所有空气一共吃掉了多少光"，但那恰恰是你正在求的未知量——死循环，而且误差会像滚雪球一样指数爆炸；从远往近算，你只需要在远端锚定一个合理假设，然后每一步都踩在已经算过的路上，稳稳当当。

假设距离间隔都是 0.1 km，用梯形面积近似：

| 当前距离  | 从当前距离累加到 500 m 的面积 | 怎么理解                    |
|-------|--------------------|-------------------------|
| 100 m | 0.961              | 从 100 m 到 500 m 的回波累计影响 |
| 200 m | 0.691              | 从 200 m 到 500 m 的累计影响   |
| 300 m | 0.427              | 从 300 m 到 500 m 的累计影响   |
| 400 m | 0.179              | 从 400 m 到 500 m 的累计影响   |
| 500 m | 0.000              | 参考点本身，不再往后累加            |

12.4.4 完整 C++ 示例

这段代码就是上面那张表的计算过程。它不是生产级大气反演代码，而是教学版：目的只是让你看清楚“数据一步一步怎么变”。

```
#include <stdio>

// 教学版 Klett 反演：看懂数据怎么一步步变化
int main() {
 const double S = 50.0; // 消光-散射比例，单位 sr

 // 距离，单位 km。0.1 km = 100 m
 const double ranges_km[] = {0.1, 0.2, 0.3, 0.4, 0.5};
 const int n = 5;

 // 假设已经扣掉背景噪声后的原始回波（单调下降，物理自治）
 const double raw_power[] = {300, 60, 32, 13, 6};

 // 远端参考点：这里取 500 m
 const int ref_index = n - 1;
 const double beta_ref = 0.0035; // 参考点后向散射，单位 km-1 sr-1

 // 第一步：距离平方校正，得到 X = P * R2
 double rcs[5];
 for (int i = 0; i < n; ++i) {
 rcs[i] = raw_power[i] * ranges_km[i] * ranges_km[i];
 }

 // 用梯形法计算：从第 i 个距离点累加到参考点的面积
 auto area_from_current_to_ref = [&](int i) -> double {
 double area = 0.0;
 for (int j = i; j < ref_index; ++j) {
 double dr = ranges_km[j + 1] - ranges_km[j];
 area += (rcs[j] + rcs[j + 1]) / 2.0 * dr;
 }
 return area;
 };

 double x_ref = rcs[ref_index];

 printf("距离(m) 原始P RCS_X 累计面积 分母 beta alpha\n");

 for (int i = 0; i < n; ++i) {
 double area = area_from_current_to_ref(i);

 // Klett 反演的核心：参考点 + 路径累计影响
 double denominator = x_ref / beta_ref + 2.0 * S * area;
 double beta = rcs[i] / denominator;
 double alpha = S * beta;

 printf("%6.0f %5.0f %6.2f %7.3f %7.1f %8.5f %7.3f\n",
 ranges_km[i] * 1000.0,
 raw_power[i],
 rcs[i],
```

```
 area,
 denominator,
 beta,
 alpha);
 }

 return 0;
}
```

运行后会得到类似这样的结果：

| 距离    | 原始 P | RCS X | 累计面积  | 分母    | $\beta$ | $\alpha = S\beta$ |
|-------|------|-------|-------|-------|---------|-------------------|
| 100 m | 300  | 3.00  | 0.961 | 524.7 | 0.00572 | 0.286             |
| 200 m | 60   | 2.40  | 0.691 | 497.7 | 0.00482 | 0.241             |
| 300 m | 32   | 2.88  | 0.427 | 471.3 | 0.00611 | 0.306             |
| 400 m | 13   | 2.08  | 0.179 | 446.5 | 0.00466 | 0.233             |
| 500 m | 6    | 1.50  | 0.000 | 428.6 | 0.00350 | 0.175             |

现在从左到右看，数据经历了这些变化：

原始回波 P

↓ 距离平方校正

RCS:  $X = P \times R^2$

↓ 加入远端参考点 beta\_ref

累计路径影响

↓ Klett 递推

后向散射 beta

↓  $\alpha = S \times \beta$

消光 alpha

最重要的结论是：

- 原始回波  $P$  一路单调下降（300→60→32→13→6），300 m 的污染团几乎被  $1/R^2$  埋没，肉眼根本看不出来。
- 做完距离平方校正后， $X$  的大趋势仍是越远越低（3.00→2.40→2.88→2.08→1.50），但 300 m 出现了一个**局部凸起**（2.88 比前后点都高，但仍低于 100 m 的 3.00）——污染增强这才露出头来。
- 做完简化 Klett 反演后，300 m 的消光  $\alpha = 0.306$  仍然最高，而且把传播衰减也一并考虑了进去。
- 所以这条射线上，300 m 附近最像真正的颗粒物增强区。

这就是 Klett 反演的价值：

它不是只看哪里亮，而是把距离影响、路径衰减和参考点一起考虑进去，再判断哪里真正浑浊。

## 12.5 离散化后程序到底怎么做

论文喜欢写连续距离  $R$ ，但真实程序不会真的在连续空间里算。程序会把距离切成很多小格子，每个小格子英文叫 `bin`。

这里 `bin` 可以翻译成：

距离小格子。

例如一条 0-600 m 的路径，可以切成这样：

雷达 | 0-100 | 100-200 | 200-300 | 300-400 | 400-500 | 500-600 |  
近端远端

程序真正做的是：

- 1. 先在远端 500-600 m 选一个参考格子。
- 2. 给这个参考格子一个可信的起始值。
- 3. 用第 6 格推第 5 格，再推第 4 格，一直推回雷达附近。

一段非常简化的假数据：

| 距离小格子     | 校正后回波 $X$ | 程序动作         |
|-----------|-----------|--------------|
| 500-600 m | 90        | 参考区段，先给起点    |
| 400-500 m | 110       | 从参考区段往回推     |
| 300-400 m | 180       | 可能有污染团，结果会升高 |
| 200-300 m | 160       | 继续往回推        |
| 100-200 m | 120       | 得到近端结果       |

### "从参考区段往回推"到底是怎么做计算的？一格一格演示给你看

这是 12.5 节最关键的问题。上面表格里写了"从参考区段往回推"，但没说**具体怎么算**。下面用真实数字一步步走一遍。

#### 准备条件（这些在开始前都已经有了）

| 已知量                       | 值      | 怎么来的                         |
|---------------------------|--------|------------------------------|
| 消光-散射比 $S$                | 50 sr  | 经验参数，事先设好                    |
| 参考区段 $\beta_{\text{ref}}$ | 0.0020 | 假设 500-600 m 是干净空气，查标准大气模型得到 |

| 已知量                   | 值      | 怎么来的         |
|-----------------------|--------|--------------|
| 参考区段 $X_{\text{ref}}$ | 90     | 设备测到的、校正后的回波 |
| 每格距离宽度 $\Delta R$     | 0.1 km | 采样间隔         |

由此算出参考区段的"分母种子":

$$\text{种子} = \frac{X_{\text{ref}}}{\beta_{\text{ref}}} = \frac{90}{0.0020} = 45000$$

这个种子值在整个反演过程中是**固定的**，所有格子都要用到它。

第 1 步：算 400-500 m 这格（第一格往回推）

现在你要算 400-500 m 这格的  $\beta$  和  $\alpha$ 。你手上有：

- 1. 这格的回波  $X_{400} = 110$ （设备测的）
- 2. 种子值 = 45000（上面算好的）
- 3. 路径积分项（从 400 m 到 500 m 之间的累计面积）

路径积分用梯形法：两端是  $X_{400} = 110$  和  $X_{500} = 90$ ，宽度  $\Delta R = 0.1$  km：

$$\text{面积}_{400 \rightarrow 500} = \frac{110 + 90}{2} \times 0.1 = 10$$

现在套 Klett 公式：

$$\beta_{400} = \frac{X_{400}}{\text{种子} + 2S \times \text{面积}_{400 \rightarrow 500}} = \frac{110}{45000 + 2 \times 50 \times 10} = \frac{110}{46000} \approx 0.00239$$

再用  $\alpha = S \times \beta$  算消光：

$$\alpha_{400} = 50 \times 0.00239 \approx 0.120 \text{ km}^{-1}$$

怎么理解这一步的结果？

- 400 m 处的回波（110）比参考区段（90）大一些。
- 算出来的  $\beta_{400} = 0.00239$  也比  $\beta_{\text{ref}} = 0.0020$  大一些。
- 合理——回波变强了，说明这一格比远处更浑浊。

第 2 步：算 300-400 m 这格（再往回推一格）

这格的回波  $X_{300} = 180$ ，比上一格大很多。

路径积分现在要从 300 m 一直累加到 500 m（跨两格）：

面积<sub>300→500</sub> =  $\frac{180 + 110}{2} \times 0.1 + \frac{110 + 90}{2} \times 0.1 = 14.5 + 10 = 24.5$

注意：第二段面积（400→500 = 10）在上一步已经算过了，这里直接复用，不用重算。

套公式：

$$\beta_{300} = \frac{180}{45000 + 2 \times 50 \times 24.5} = \frac{180}{45000 + 2450} = \frac{180}{47450} \approx 0.00379$$
$$\alpha_{300} = 50 \times 0.00379 \approx 0.190 \text{ km}^{-1}$$

怎么理解？

- 300 m 处回波骤增到 180，是所有格子里最大的。
- 算出来  $\beta_{300} = 0.00379$ ，是 400 m 的 1.6 倍。
- 这说明 300 m 附近确实有一个污染增强区——和我们的预期一致。

第 3 步：算 200-300 m 这格

路径积分累加 200→300、300→400、400→500 三段：

面积 =  $\frac{160 + 180}{2} \times 0.1 + 14.5 + 10 = 17 + 24.5 = 41.5$

$$\beta_{200} = \frac{160}{45000 + 2 \times 50 \times 41.5} = \frac{160}{49150} \approx 0.00326$$
$$\alpha_{200} = 50 \times 0.00326 \approx 0.163 \text{ km}^{-1}$$

回波降回到 160， $\beta$  也跟着降下来。

第 4 步：算 100-200 m 这格

面积 =  $\frac{120 + 160}{2} \times 0.1 + 41.5 = 14 + 41.5 = 55.5$

$$\beta_{100} = \frac{120}{45000 + 2 \times 50 \times 55.5} = \frac{120}{50550} \approx 0.00237$$
$$\alpha_{100} = 50 \times 0.00237 \approx 0.119 \text{ km}^{-1}$$

算完后的完整结果：

| 距离        | 回波 $X$ | $\beta$      | $\alpha = S\beta$ | 看到了什么    |
|-----------|--------|--------------|-------------------|----------|
| 500-600 m | 90     | 0.00200 (假设) | 0.100             | 参考点，干净空气 |
| 400-500 m | 110    | 0.00239      | 0.120             | 轻微升高     |

| 距离        | 回波 $X$ | $\beta$ | $\alpha = S\beta$ | 看到了什么     |
|-----------|--------|---------|-------------------|-----------|
| 300-400 m | 180    | 0.00379 | 0.190             | 峰值——污染增强区 |
| 200-300 m | 160    | 0.00326 | 0.163             | 开始回落      |
| 100-200 m | 120    | 0.00237 | 0.119             | 接近背景      |

回到你的问题：回波和  $\beta$  之间到底什么关系？

你问的核心是：“知道 400-500 m 的回波，怎么算出它的  $\alpha$  和  $\beta$ ？”答案就在上面的第 1 步：

- 1. 回波  $X$  出现在**分子**里：回波越大， $\beta$  越大。
- 2. 但分母里还有一个**路径积分项**：这一格离参考点越远（路径越长），分母越大， $\beta$  会被压低一点。
- 3. 所以  $\beta$  不是简单地等于“回波大就是  $\beta$  大”，而是**回波 ÷（参考种子 + 路径累计影响）**。

形象地说：

text  $\beta$  = 这格的回波有多亮 ÷（参考点的锚 + 从这格到参考点一路累计的“遮挡账单”）

回波告诉你“这一格亮不亮”，分母告诉你“为了看到这一格，光在路上付出了多少代价”。两者一除，才是这一格真正的浑浊程度。

12.6 消光-散射比例对结果影响有多大

lidar ratio 这个英文建议直接在脑子里替换成：

消光-散射比例。

它是一个很敏感的经验参数。为什么敏感？因为算法要靠它把“打回来多少”换算成“吃掉多少”。

同一条回波，如果你选不同的  $S$ ，推出来的消光和 PM 可能会不一样。

假设同一个位置估出来的  $\beta = 0.004$ ：

| 假设的 $S$ | 推出来的 $\alpha = S \times \beta$ | 结果倾向        |
|---------|--------------------------------|-------------|
| 30      | 0.12                           | 偏保守，PM 可能偏低 |
| 50      | 0.20                           | 中间值         |
| 70      | 0.28                           | 偏激进，PM 可能偏高 |

这就是为什么工程系统不会随便写死一个值。常见做法是：

- 1. 先用文献或厂商经验给一个范围。
- 2. 再拿本地地面 PM 站的数据反复校准。



3. 必要时按天气、季节、污染来源分别取值。

对入门者来说，只要先记住一句：

$S$  选得大，消光和 PM 往往会被推高； $S$  选得小，结果往往会被压低。

 **你的疑问完全正确——不同物质确实应该有不同的  $S$ ，全程用同一个 50 是简化**

你敏锐地抓住了简化 Klett 的一个核心局限。下面把这件事说透。

---

**先确认：你的直觉是对的**

$S = \alpha/\beta$ ，也就是"吃掉多少光 ÷ 弹回来多少光"。不同物质吃光和弹光的能力完全不同：

| 物质类型          | 典型 $S$<br>(sr) | 为什么是这个值                               |
|---------------|----------------|---------------------------------------|
| 纯净空气分子        | 8.3            | 分子几乎不吸收光，只 Rayleigh 散射，"吃"得少所以 $S$ 很小 |
| 城市雾霾（硫酸盐+硝酸盐） | 40-60          | 有吸收有散射，中等                             |
| 沙尘 / 扬尘       | 40-70          | 颗粒大、形状不规则，吸收较多                        |
| 烟尘 / 生物质燃烧    | 60-100         | 含黑碳，吸收极强， $S$ 最大                      |
| 水云滴           | 15-25          | 近乎纯散射，几乎不吸收                           |

所以如果你 500 m 处是干净空气，300 m 处是扬尘团，它们的真实  $S$  可能差 5 倍以上。

---

**那 12.5 节全程用  $S = 50$  为什么"还能算"？**

因为 12.5 节用的是 "**固定 lidar ratio**" 的简化 Klett——这是入门教学版，目的是让你先理解递推流程，而不是追求绝对精度。固定  $S$  的好处是：

- 只有一个未知量，公式简单，能一行算出来。
- 远端参考点选得好时，近场结果的**形状**（哪里有峰、哪里有谷）大致是对的。

它的代价是：**绝对值会偏**。如果某段空气的真实  $S$  是 70 而你用了 50，那里的消光会被低估约 30%。

---

**进阶做法：让  $S$  随距离变化（可变 lidar ratio）**

真实工程系统不会全程写死一个  $S$ 。常见做法有两类：

**做法 1：分段设不同的  $S$**

如果你大概知道哪一段是什么类型的气溶胶（比如靠偏振通道判断某层是沙尘还是水雾），就可以给不同区段设不同的  $S$ ：

100-200 m  $S = 60$  （城市背景气溶胶）  
200-300 m  $S = 70$  （检测到沙尘层，偏高）  
300-400 m  $S = 50$  （普通雾霾）  
400-500 m  $S = 55$  （过渡区）  
500-600 m  $S = 8.3$  （干净空气，纯分子散射）

这样在 Klett 递推时，每跨过一段边界就切换  $S$  值，比全程用 50 更接近真实。

### 做法 2：Fernald 双分量分离

Fernald 比 Klett 多做了一步——它把总消光拆成两部分：

总后向散射 = 分子贡献 + 气溶胶贡献  
总消光 = 分子消光( $S_{\text{mol}}=8.3$ ) + 气溶胶消光( $S_{\text{aer}}=50$ )

分子的  $S_{\text{mol}} = 8.3$  是精确已知的（纯物理常数级别），不用猜。需要假设的只有气溶胶的  $S_{\text{aer}}$ ，而这个值可以从文献、偏振信息或地面标定中获得。所以 Fernald 在理论上比固定- $S$  的 Klett 更严谨。

### 那为什么不直接上 Fernald？

因为它更复杂——需要同时维护分子廓线和气溶胶廓线两条线，计算量更大，而且仍然需要你假设  $S_{\text{aer}}$ 。对于入门教学，先用固定  $S$  的 Klett 理解"从远到近逐格递推"这个核心流程，再进阶到 Fernald，学习曲线更平缓。

**一句话总结：**全程用  $S = 50$  是教学简化，你的直觉完全对——真实系统中分子段  $S$  约 8.3，沙尘段约 70，烟尘段甚至可达 100。进阶做法是分段设定或用 Fernald 双分量分离，但核心递推逻辑和 12.5 节演示的完全一样，只是  $S$  不再是一个常数而是一个随距离变化的数组。

### 用不同 S 把 12.5 节的 4 步重新算一遍

上面讲了"不同物质应该用不同  $S$ "，但只给了概念。下面用 12.5 节**完全相同的回波数据**，换成分段  $S$ ，一步一步算给你看，最后和固定  $S = 50$  做对比。

### 场景假设

还是这条射线，还是同样的 5 格，但这次我们假设已经通过偏振通道或其他手段知道了每段空气的类型：

| 距离小格子     | 回波 $X$ | $S$ (sr) | 判断依据       | 物质类型 |
|-----------|--------|----------|------------|------|
| 500-600 m | 90     | 8.3      | 远端干净，纯分子散射 | 参考区段 |

| 距离小格子     | 回波 $X$ | $S$ (sr) | 判断依据    | 物质类型  |
|-----------|--------|----------|---------|-------|
| 400-500 m | 110    | 55       | 过渡区     | 轻度气溶胶 |
| 300-400 m | 180    | 70       | 检测到沙尘信号 | 沙尘层   |
| 200-300 m | 160    | 50       | 普通城市雾霾  | 雾霾    |
| 100-200 m | 120    | 60       | 近地面背景   | 城市背景  |

### 准备条件

种子值不变（还是用远端参考点）：

$$\text{种子} = \frac{X_{\text{ref}}}{\beta_{\text{ref}}} = \frac{90}{0.0020} = 45000$$

参考点消光也重新算（因为  $S_{\text{ref}}$  从 50 变成了 8.3）：

$$\alpha_{\text{ref}} = S_{\text{ref}} \times \beta_{\text{ref}} = 8.3 \times 0.002 = 0.017 \text{ km}^{-1}$$

注意：这个值比固定  $S = 50$  时的 0.1 小了 6 倍——干净空气确实不怎么"吃"光。

### 关键变化：路径积分变成了"加权积分"

固定  $S$  时，路径积分是纯粹的面积累加：

$$\text{面积} = \sum \frac{X_j + X_{j+1}}{2} \times \Delta R$$

分段  $S$  时，每段面积要乘以该段的  $S$ ：

$$\text{加权面积} = \sum S_j \times \frac{X_j + X_{j+1}}{2} \times \Delta R$$

这就好比高速公路收费——固定  $S$  是"全程统一费率"，分段  $S$  是"不同路段不同费率"。沙尘段（ $S = 70$ ）就是收费最贵的那段。

### 第 1 步：算 400-500 m ( $S = 55$ )

加权面积（只有一段，从 400m 到 500m）：

$$55 \times \frac{110 + 90}{2} \times 0.1 = 55 \times 10.0 = 550$$

套公式：

$$\beta_{400} = \frac{110}{45000 + 2 \times 550} = \frac{110}{46100} \approx 0.00239$$

$$\alpha_{400} = \underbrace{55}_{\text{本格 } S} \times 0.00239 \approx 0.131 \text{ km}^{-1}$$

## 第 2 步：算 300-400 m ( $S = 70$ , 沙尘层！)

加权面积（跨两段，每段用各自的  $S$ ）：

$$\underbrace{70 \times \frac{180 + 110}{2} \times 0.1}_{S=70 \text{ 段}} + \underbrace{55 \times \frac{110 + 90}{2} \times 0.1}_{S=55 \text{ 段}} = 70 \times 14.5 + 55 \times 10.0 = 1015 + 550 = 1565$$

套公式：

$$\beta_{300} = \frac{180}{45000 + 2 \times 1565} = \frac{180}{48130} \approx 0.00374$$

$$\alpha_{300} = \underbrace{70}_{\text{沙尘段 } S} \times 0.00374 \approx 0.262 \text{ km}^{-1}$$

**注意  $\alpha$  的变化：**固定  $S = 50$  时这里算出来是 0.190，现在用  $S = 70$  算出来是 0.262——高了 38%。这就是沙尘层“吃光更狠”的直接体现。

## 第 3 步：算 200-300 m ( $S = 50$ , 普通雾霾)

加权面积（跨三段）：

$$\underbrace{50 \times 17.0}_{S=50} + \underbrace{70 \times 14.5}_{S=70} + \underbrace{55 \times 10.0}_{S=55} = 850 + 1015 + 550 = 2415$$

$$\beta_{200} = \frac{160}{45000 + 2 \times 2415} = \frac{160}{49830} \approx 0.00321$$

$$\alpha_{200} = 50 \times 0.00321 \approx 0.161 \text{ km}^{-1}$$

## 第 4 步：算 100-200 m ( $S = 60$ , 城市背景)

加权面积（跨四段，全部）：

$$\underbrace{60 \times 14.0}_{S=60} + \underbrace{50 \times 17.0}_{S=50} + \underbrace{70 \times 14.5}_{S=70} + \underbrace{55 \times 10.0}_{S=55} = 840 + 850 + 1015 + 550 = 3255$$

$$\beta_{100} = \frac{120}{45000 + 2 \times 3255} = \frac{120}{51510} \approx 0.00233$$

$$\alpha_{100} = 60 \times 0.00233 \approx 0.140 \text{ km}^{-1}$$

完整结果对比：固定  $S = 50$  vs 分段  $S$

| 距离        | $S$ | $\beta$ (固定) | $\alpha$ (固定) | $\beta$ (分段) | $\alpha$ (分段) | $\alpha$ 差异      |
|-----------|-----|--------------|---------------|--------------|---------------|------------------|
| 500-600 m | 8.3 | 0.00200      | 0.100         | 0.00200      | <b>0.017</b>  | ↓ 83% (干净空气吃光极少) |
| 400-500 m | 55  | 0.00239      | 0.120         | 0.00239      | <b>0.131</b>  | ↑ 9%             |
| 300-400 m | 70  | 0.00379      | 0.190         | 0.00374      | <b>0.262</b>  | ↑ 38% (沙尘层差异最大)  |
| 200-300 m | 50  | 0.00326      | 0.163         | 0.00321      | <b>0.161</b>  | ↓ 1% (恰好 $S$ 没变) |
| 100-200 m | 60  | 0.00237      | 0.119         | 0.00233      | <b>0.140</b>  | ↑ 18%            |

从这张表能看出什么？

- $\beta$  差异很小（都在小数点后第四位）：因为  $\beta$  主要由回波  $X$ （分子）决定，分母的变化影响不大。
- $\alpha$  差异很大：因为  $\alpha = S \times \beta$ ， $S$  变了  $\alpha$  直接等比例变。
- 差异最悬殊的是两个极端——干净空气段（ $S$  从 50 降到 8.3， $\alpha$  暴跌 83%）和沙尘段（ $S$  从 50 升到 70， $\alpha$  飙升 38%）。
- 200-300 m 几乎没变：因为恰好给这格设的  $S = 50$ ，和固定值一样。

这就回答了你的疑问：固定  $S$  不是错在递推流程上，而是错在用同一个“换算比例”去套所有物质。流程完全一样，只是每一步乘的  $S$  不同。

🤖 那这个  $S$  到底怎么来的？实际工程中怎么确定？

这是激光雷达反演最核心的问题之一。 $S = \alpha/\beta$  本质上取决于气溶胶的化学成分、形状、大小和湿度，所以它不是一个仪器常数，而是随场景变化的。业界经过几十年发展，形成了 5 种主要获取途径，从“最精确但最贵”到“最省事但最粗”排列如下：

方法一：拉曼雷达直接测量（精度最高，设备最贵）

原理：普通弹性雷达只能测到总回波  $X$ ，无法区分  $\alpha$  和  $\beta$ 。但拉曼雷达能同时接收两个信号：

- **弹性散射**（发射波长 = 接收波长）→ 包含  $\alpha$  和  $\beta$  的耦合信息
- **氮气拉曼散射**（波长偏移  $\sim 607\text{ nm}$  @  $532\text{nm}$ ）→ 因为大气中  $N_2$  浓度恒定已知，回波只受**消光**影响，不受气溶胶后散射干扰

有了独立的氮气拉曼信号，就可以**分别求解**  $\alpha(R)$  和  $\beta(R)$ ，然后直接计算  $S(R) = \alpha(R)/\beta(R)$ 。

**类比：**普通弹性雷达像"只给你总重量，不告诉你多少是脂肪多少是肌肉"；拉曼雷达像"同时给了体脂率和肌肉量，BMI (= S) 自然就算出来了"。

**精度：** $S$  的不确定度可控制在  $\pm 5 \sim 10\text{ sr}$  以内，是目前公认的金标准。

**代价：**拉曼信号比弹性信号弱 **3~4 个数量级**（因为拉曼散射截面极小），需要：

- 更大功率的激光器
- 更大口径望远镜
- 更长积分时间（夜间典型  $30\text{ min} \sim 2\text{ h}$ ）
- 昂贵的多通道光谱分光系统

**代表性论文：**

- **Ansmann et al. (1992)**, "*Calibration of 532-nm aerosol backscatter and extinction from Raman lidar*", Applied Optics — **拉曼雷达反演的开山之作**，首次提出利用  $N_2$  拉曼回波独立求解消光系数
- **Ansmann et al. (1990)**, "*Lidar observations of the stratospheric aerosol layer*" — SPICE 实验验证

**业界使用：**欧洲 **EARLINET** (European Aerosol Research Lidar Network) 的近 30 个站点配备拉曼通道，用于校准和建立  $S$  气候学数据库

## 方法二：太阳光度计协同 (AERONET, 性价比最高)

**原理：**AERONET（全球地基太阳光度计网络，500+ 站点）可以独立测量**大气柱积分气溶胶光学厚度** AOD (Aerosol Optical Depth)。同时，激光雷达提供  $\beta(R)$  的**高度分辨剖面**。将两者结合：

$$\text{AOD} = \int_0^{z_{\text{top}}} \alpha(R) dR = \int_0^{z_{\text{top}}} S \cdot \beta(R) dR$$

对整条雷达剖面用一个  $S$ （或分段用几个  $S$ ），调节  $S$  的值，使得积分消光  $\int S \cdot \beta dR$  与 AERONET 测量的 AOD **匹配**。这就反推出了  $S$ 。

**类比：**太阳光度计告诉你"整栋楼一共用了多少电" (AOD)，雷达告诉你"每层楼的用电量分布" ( $\beta$  剖面)，你通过调"每度电单价" ( $S$ ) 让总电费对上。

**精度：** $\pm 10 \sim 20\text{ sr}$ （不如拉曼精确，但远好于纯猜）。

**优点：**AERONET 数据**全球免费开放**，覆盖面广；太阳光度计是被动仪器，便宜、免维护、自动运行。

**局限：**只能得到**柱平均**  $S$ ，无法给出每一高度的  $S$ ；需要晴空条件；假设气溶胶类型在整柱内比较均匀。

代表性论文：

- Müller et al. (2007), "Aerosol-type-dependent lidar-ratio observed with Raman lidar and constraints by Sun photometer measurements" — 联合 EARLINET 和 AERONET 建立  $S$  查找表
- Cattrall et al. (2005), "Observations of aerosol single-scattering albedo at different sites around the globe using spaceborne CALIPSO lidar and AERONET Sun photometer data" — 协同反演

方法三：文献查找表（查手册法，最快速）

如果你没有拉曼雷达、也没有 AERONET 站点在附近，最常用的办法就是**查表**。科学界经过几十年测量，已经积累了大量  $S$  的经验值：

| 气溶胶类型     | $S$ 典型值 (sr) @ 532 nm | 来源                    |
|-----------|-----------------------|-----------------------|
| 纯净海洋性     | 20~25                 | EARLINET 海岸站          |
| 沙尘        | 40~60                 | Sahara 沙尘暴观测          |
| 生物质燃烧烟尘   | 60~80                 | 亚马逊、非洲干季              |
| 都市污染/煤烟   | 50~70                 | EARLINET 城市站          |
| 清洁大陆性     | 30~50                 | 欧洲/北美背景值              |
| 火山灰       | 40~60                 | Eyjafjallajökull 2010 |
| 纯净大气 (分子) | $8.3 \approx 8\pi/3$  | 理论计算                  |

**做法：** 根据观测地点、季节、HYSPLIT 后向轨迹分析气溶胶来源，然后从上表（或更详细的文献）选取最匹配的  $S$ 。

代表性数据库/论文：

- EARLINET climatology database (公开, <https://earlinet.org>)
- Sakai et al. (2003), "Lidar ratio of tropospheric aerosols measured by Raman lidar" — 多种类型  $S$  汇总
- Omar et al. (2009), "The CALIPSO Automated Aerosol Classification and Lidar Ratio Selection Algorithm" — CALIPSO 卫星使用的  $S$  查找表

方法四：卫星业务化假定（全球覆盖，精度粗）

**以 CALIPSO 卫星为例：** NASA/CNES 的 CALIPSO (2006-2023) 搭载正弹性雷达，没有拉曼通道，无法自己测量  $S$ 。它的做法是先用**场景分类算法** (Scene Classification Algorithm, SCA) 判断每一层的气

溶胶类型，然后为每种类型**预设一个固定  $S$** ：

| CALIPSO 分类           | 假定 $S$ (sr) @ 532 nm |
|----------------------|----------------------|
| Clean Continental    | 35                   |
| Polluted Continental | 70                   |
| Clean Marine         | 20                   |
| Dust                 | 40                   |
| Polluted Dust        | 55                   |
| Smoke                | 70                   |
| PSC (Type 1a/1b)     | 25~35                |

**优点：** 全球覆盖，标准化处理，产品可追溯。

**缺点：** 分类算法有时判错（比如把沙尘误判为污染），一旦类型错则  $S$  也错，误差可超 50%。

**代表论文：**

- Omar et al. (2009), JGR — CALIPSO 的  $S$  选择算法完整描述
- Kim et al. (2018), "*Calibration of CALIPSO lidar using the Cloud-Aerosol Lidar with Orthogonal Polarization (CALIOP)*" — CALIPSO v4 校准改进

### 方法五：迭代约束法（科研级精细处理）

**原理：** 如果你有地面能见度传感器、浊度计或颗粒物监测站（PM2.5/PM10）的独立测量，可以：

1. 先假设一个  $S_0$ （比如查表得到 50 sr）
2. 用  $S_0$  跑反演得到  $\alpha$  剖面
3. 将反演结果在近地面与能见度/PM 数据比较
4. 计算残差  $\Delta$ ，调整  $S$  使  $\Delta$  最小化
5. 重复 2-4 直到收敛

这相当于把  $S$  当成未知参数，用额外的约束条件来优化。

**代表方法：**

- Sasano & Nakane (1984) — 提出利用已知消光边界值反推  $S$  的方法
- Veselovskii et al. (2002) — 结合多波长雷达和拉曼信号迭代优化  $S$



### 五种方法对比总览

| 方法        | 精度                       | 成本                | 覆盖范围         | 典型应用              |
|-----------|--------------------------|-------------------|--------------|-------------------|
| ① 拉曼雷达    | ★★★★★ $\pm 5$ sr         | 极高（数百万设备）         | 单点           | 科研金标准、EARLINET 校准 |
| ② 太阳光度计协同 | ★★★★ $\pm 10 \sim 15$ sr | 中等（免费 AERONET 数据） | AERONET 站点附近 | 地基雷达日常处理          |
| ③ 文献查找表   | ★★★ $\pm 20$ sr          | 零成本               | 全球适用         | 工程项目、快速估算         |
| ④ 卫星假定值   | ★★ $\pm 30$ sr           | 数据免费              | 全球覆盖         | CALIPSO 业务产品      |
| ⑤ 迭代约束法   | ★★★★ $\pm 10$ sr         | 需辅助传感器            | 单点           | 有地面监测的站点          |

**对你的项目的建议：** 如果你做的是城市/区域污染监测，最实用的路线是——先用方法③（查表选  $S \approx 50 \sim 70$  for 都市污染），有条件时用方法②（找最近的 AERONET 站点做 AOD 约束验证）。方法①拉曼雷达通常是大型科研站点的配置，工程项目较少使用。

### 12.7 为什么反演前后都要平滑

LiDAR 回波里一定有噪声。噪声有时只是一个小尖峰，但在反演算法里可能被放大。所以反演前后经常要平滑。平滑不是为了把图修得漂亮，而是为了让算法不要被单个噪声点带偏。看一个假数据：

| 距离    | 原始结果 | 平滑后 | 说明          |
|-------|------|-----|-------------|
| 100 m | 42   | 43  | 正常          |
| 200 m | 45   | 46  | 正常          |
| 300 m | 120  | 67  | 可能是噪声尖峰，被压下 |
| 400 m | 50   | 55  | 正常          |
| 500 m | 52   | 53  | 正常          |

但平滑也不能太狠。太狠会把真实污染团的边界抹掉。一个简单判断标准：

- 1. 如果只是孤立一个点特别高，多半要压一压。

- 2. 如果连续好几格都高，可能是真实污染团，不应该抹掉。
- 3. 如果热点边界被磨得看不清，说明平滑窗口太大。

🤖 那具体怎么平滑？用了什么算法？

好问题。平滑不是随手"拉一拉"就行的。业界常用的平滑算法有 **4 种**，从最简单到最精细排列：

算法一：滑动平均 (Moving Average, 最简单)

取当前点和左右各  $w$  个邻居，直接算算术平均。窗口宽度  $w$  越大越平滑，但细节丢得越多。

公式：

$$\hat{X}_i = \frac{1}{2w + 1} \sum_{k=-w}^w X_{i+k}$$

**举例：** 假设原始 RCS 序列  $X = [120, 160, 180, 110, 90]$ （还是我们的老朋友数据），用  $w = 1$ （即 3 点窗口）做滑动平均：

- $\hat{X}_1 = \frac{120+160+180}{3} = \frac{460}{3} \approx 153$
- $\hat{X}_2 = \frac{160+180+110}{3} = \frac{450}{3} = 150$
- $\hat{X}_3 = \frac{180+110+90}{3} = \frac{380}{3} \approx 127$

| 位置    | 原始 $X$ | 平滑后 $\hat{X} (w = 1)$ |
|-------|--------|-----------------------|
| 100 m | 120    | 153                   |
| 200 m | 160    | 150                   |
| 300 m | 180    | 127                   |
| 400 m | 110    | — (边缘, 不够窗口)          |
| 500 m | 90     | — (边缘)                |

**优点：** 实现最简单，一行代码搞定。

**缺点：** 对所有点一视同仁，会把真实尖峰（比如薄污染层）也压平；边缘点没有足够邻居，需要特殊处理。

算法二：中值滤波 (Median Filter, 抗尖峰最强)

不取平均，而是取窗口内的**中位数**。这样即使有一个极端噪声尖峰，也不会拉偏结果。

**举例：** 假设有一组含噪声的数据  $X = [100, 105, \mathbf{500}, 110, 108]$ ，其中 500 是噪声尖峰：

- **滑动平均** ( $w = 1$ ) :  $\hat{X}_3 = \frac{105+500+110}{3} = 238 \rightarrow$  尖峰把平均值拉飞了
- **中值滤波** ( $w = 1$ ) : 排序  $[105, 500, 110] \rightarrow [105, 110, 500]$ ，取中间值 = 110  $\rightarrow$  尖峰被完全剔除！

| 位置    | 原始 $X$     | 滑动平均       | 中值滤波       |
|-------|------------|------------|------------|
| 100 m | 100        | —          | —          |
| 200 m | 105        | 235        | 105        |
| 300 m | <b>500</b> | <b>238</b> | <b>110</b> |
| 400 m | 110        | 239        | 108        |
| 500 m | 108        | —          | —          |

**适用场景：** 回波里有偶尔的电子尖峰噪声（如探测器暗计数突跳），中值滤波可以"一刀切掉"而不伤周围数据。

### 算法三：Savitzky-Golay 滤波器（最精细，保留形状）

这是激光雷达领域**最常用**的高阶平滑方法。它不是简单取平均，而是在窗口内**拟合一条多项式曲线**（通常是 2 次或 3 次），然后用拟合曲线上的值替换原始值。

**原理一句话：** "我假设这一小段数据应该是一条平滑的抛物线，然后找到最贴合的抛物线，用抛物线上的值。"

**为什么比滑动平均好？** 因为多项式拟合能**保留真实的峰值和谷值**——如果一个尖峰是"抛物线形的"（像真实污染层那样中间高两边低），Savitzky-Golay 会保留它；而滑动平均会把它压平。

**关键参数：**

- 窗口宽度  $w$ （一般 5~15 个距离bin，即 50~150 m）
- 多项式阶数  $p$ （一般 2~3 阶）

**经验法则：**  $w$  越大越平滑； $p$  越高越贴合细节但也越容易被噪声带偏。典型选择是  $w = 7, p = 2$  或  $w = 11, p = 3$ 。

**代码实现 (C++)：**

```
#include <vector>

// Savitzky-Golay 滤波 (window_length=7, polyorder=2)
// 固定参数的系数是预计算好的，直接做加权卷积：
```

```
// 系数 = [-2, 3, 6, 7, 6, 3, -2] / 21
std::vector<double> savgol_filter(const std::vector<double>& X) {
 const double c[] = {-2, 3, 6, 7, 6, 3, -2};
 const int half = 3; // 窗口半径 = (7 - 1) / 2
 const int n = (int)X.size();
 std::vector<double> out(n, 0.0);

 for (int i = 0; i < n; ++i) {
 double sum = 0.0;
 for (int k = -half; k <= half; ++k) {
 int idx = i + k;
 if (idx < 0) idx = -idx; // 边界镜像延拓
 if (idx >= n) idx = 2 * (n - 1) - idx;
 sum += X[idx] * c[k + half];
 }
 out[i] = sum / 21.0;
 }
 return out;
}
```

**业界使用：**几乎所有正规激光雷达数据处理软件（如 EARLINET 的 SCC 单计算链、Raymetrics 软件）都默认使用 Savitzky-Golay 或其变体。

**算法四：高斯加权平滑（Gaussian Smoothing，最自然）**

用一个高斯函数（钟形曲线）做权重——离中心越近的点权重越大，越远的点权重越小。

**公式：**

$$\hat{X}_i = \sum_{k=-w}^w X_{i+k} \cdot \frac{1}{\sigma \sqrt{2\pi}} e^{-k^2/(2\sigma^2)}$$

**类比：**滑动平均是"远近亲疏一视同仁"，高斯平滑是"近处的意见更重要"。

**适用场景：**信号本身在物理上就应该连续平滑变化（大气消光确实如此），高斯核更符合物理直觉。

**四种算法对比**

| 算法             | 抗尖峰   | 保留峰值形状 | 实现难度 | 典型用途     |
|----------------|-------|--------|------|----------|
| 滑动平均           | ★★    | ★      | ★    | 快速原型验证   |
| 中值滤波           | ★★★★★ | ★★     | ★★   | 剔除电子尖峰   |
| Savitzky-Golay | ★★★   | ★★★★★  | ★★★  | 雷达反演标准做法 |

| 算法   | 抗尖峰 | 保留峰值形状 | 实现难度 | 典型用途    |
|------|-----|--------|------|---------|
| 高斯加权 | ★★★ | ★★★★   | ★★   | 信号物理连续时 |

🤖 那为什么不直接发射多次激光取平均？不就自然没噪声了吗？

这是一个非常自然的想法，而且答案是：**你说的对——工程上确实会这么做！这叫“累积平均”（Accumulation / Shot Averaging），是降低噪声的第一道防线。但它不够用，还需要配合空间平滑。**原因有三层：

**原因一：时间平均 ≠ 空间平滑，它们对付的噪声不同**

- **多次发射平均（时间域）：** 激光每发射一次（叫一“发” / shot），大气状态、电子噪声都是随机的。发 1000 次取平均，随机噪声按  $\sqrt{N}$  规律降低——1000 发平均后噪声降为原来的  $1/\sqrt{1000} \approx 1/32$ ，即降低约 **30 dB**。

$\$信噪比 \propto \sqrt{N_{shots}}\$$

- **空间平滑（距离域）：** 即使时间平均做完了，某一个距离bin上的信号仍然可能有异常（比如局部云团遮挡、飞鸟、电子串扰）。空间平滑是在**同一条剖面内**沿距离方向做的。

**类比：**

- 时间平均 = “拍 1000 张照片叠在一起去噪”（去的是时间随机噪声）
- 空间平滑 = “在一张照片上用美颜磨皮”（去的是空间上的局部毛刺）

**两个都要做，不能互相替代。**

**原因二：时间分辨率和空间分辨率互相矛盾**

你不能无限增加发射次数——每多一发，测量时间就变长，大气也在变化。

| 累积发数     | 时间分辨率  | 噪声降低   | 问题             |
|----------|--------|--------|----------------|
| 1 发（单脉冲） | ~0.1 秒 | 无降低    | 噪声极大，几乎不可用     |
| 100 发    | ~10 秒  | ↓ 10×  | 较好，但仍有残余噪声     |
| 1000 发   | ~100 秒 | ↓ 32×  | 信噪比较好          |
| 10000 发  | ~17 分钟 | ↓ 100× | 大气已经变了，时间分辨率太差 |

**典型折中：** 业务激光雷达一般用 **1000~5000 发** 累积（约 1~5 分钟），再配合 **空间平滑** 进一步降噪。光靠堆发数会把时间分辨率拖垮。

### 原因三：有些噪声不是随机的，平均消不掉

时间平均只能消除**随机噪声**（符合正态分布的那种）。但实际中还有：

- **系统噪声（1/f 噪声）：** 低频漂移，平均反而会把它"锁定"
- **探测器非线性：** 强信号时响应非线性，多次平均不能修正
- **背景光干扰：** 白天的太阳光散射是连续的，不随发射次数随机变化
- **云层/飞鸟瞬态干扰：** 只在某一发出现，但如果恰好被包含在累积窗口里，会偏移平均值

对于这些，需要中值滤波（剔除瞬态干扰）或差分处理（扣除背景），光靠多发平均是不够的。

### 完整的降噪流程（实际工程中的标准做法）

单发回波

|

▼

① 累积平均（1000~5000 发，时间域去随机噪声）

|

▼

② 背景扣除（减去探测器暗噪声 + 天光背景）

|

▼

③ 距离校正 ( $X = P \times R^2$ )

|

▼

④ 空间平滑（Savitzky-Golay，距离域去局部毛刺）← 反演前

|

▼

⑤ Fernald/Klett 反演

|

▼

⑥ 结果平滑（轻量 Savitzky-Golay 或中值，反演后微调）← 反演后

|

▼

最终  $\alpha(R)$  剖面

**关键：** ④在反演**前**平滑原始信号，⑥在反演**后**平滑结果。这就是 12.7 标题所说的"反演前后都要平滑"。

## 12.8 湿度修正为什么必须进入主流程

这一节非常重要，因为湿度会让初学者误判。

很多颗粒物会吸水。空气越潮，颗粒物表面吸的水越多，散射就越强。于是 LiDAR 看到的亮度可能变高。但这不一定代表“干颗粒物质量真的变多了”。



用一句话说：

高湿度会把颗粒物“看起来变胖”，所以光学信号会变强。

假设同样一团干颗粒物，湿度不同：

| 相对湿度 RH | LiDAR 看到的光学强度 | 如果不修正  | 修正后理解     |
|---------|---------------|--------|-----------|
| 40%     | 100           | 正常     | 接近干颗粒物本身  |
| 70%     | 140           | 可能误判偏浓 | 有一部分是湿度放大 |
| 90%     | 220           | 很容易误报警 | 必须谨慎修正或降权 |

工程上常用一个湿度修正函数，把“湿空气下变亮的部分”尽量扣回来。公式可以先不用背，只记住它在做这件事：

干态光学量 = 湿态光学量 × 湿度修正系数

这里的“干态”不是说空气真的被烘干了，而是说：

算法尽量把结果换算成更接近干颗粒物质量的样子。

🤖 那湿度修正到底怎么算？是直接乘以湿度百分比吗？系数是多少？

不是直接乘以湿度。湿度修正的核心是一个**非线性函数**，而且系数不是固定的——它取决于颗粒物**什么成分**。下面一步步拆解。

第一步：定义“散射增强因子” f(RH)

先定义一个关键量：**散射增强因子**  $f(RH)$ ，表示湿空气下光学信号比干空气下放大了多少倍：

$$f(RH) = \frac{\text{湿度 } RH \text{ 下的光学信号}}{\text{干态}(RH_{ref}) \text{ 下的光学信号}}$$

规定  $f(RH_{ref}) = 1$ （干态参考点不放大）。湿度修正就是：

$$\text{干态光学量} = \frac{\text{湿态实测光学量}}{f(RH)}$$

所以你看到的"修正系数"其实就是  $1/f(RH)$ 。关键问题变成： $f(RH)$  怎么算？

## 第二步：业界最常用的两个公式

### 公式 A: $\gamma$ 指数模型 (Hänel 模型, 工程上最常见)

$$f(RH) = \left( \frac{1 - RH_{ref}}{1 - RH} \right)^\gamma$$

其中：

- $RH$  是相对湿度 (用小数, 如 0.7 而不是 70%)
- $RH_{ref}$  是参考干态湿度, 一般取 0.3~0.4 (即 30%~40%)
- $\gamma$  (伽马) 是一个经验参数, 取决于颗粒物成分

| 气溶胶类型     | $\gamma$ 典型范围 | 吸湿性强弱    |
|-----------|---------------|----------|
| 海盐        | 0.5~0.9       | 极强 (最吸水) |
| 硫酸铵       | 0.3~0.5       | 强        |
| 硝酸铵       | 0.3~0.6       | 强        |
| 城市污染 (混合) | 0.2~0.4       | 中等       |
| 生物质燃烧     | 0.1~0.25      | 弱        |
| 沙尘        | 0.05~0.15     | 很弱       |
| 黑碳/煤烟     | 0.0~0.05      | 几乎不吸水    |

**为什么这个公式长这样?** 当  $RH \rightarrow 1.0$  (100%湿度) 时, 分母  $(1 - RH) \rightarrow 0$ , 整个分数  $\rightarrow \infty$ , 所以  $f \rightarrow \infty$ ——意味着接近饱和湿度时光学信号可以放大很多倍。这符合物理观测: 高湿度下颗粒物大量吸水, 体积膨胀, 散射急剧增强。

### 公式 B: $\kappa$ 模型 (Petters & Kreidenweis 2007, 科研标准)



$$f(RH) = 1 + \kappa \cdot \frac{RH}{1 - RH}$$

其中  $\kappa$ （卡帕）是**单一吸湿性参数**，物理含义更明确：

| 气溶胶类型   | $\kappa$ 典型值 | 说明   |
|---------|--------------|------|
| 海盐      | 1.0~1.5      | 最强吸湿 |
| 硫酸铵     | 0.5~0.6      | 强    |
| 硝酸铵     | 0.5~0.7      | 强    |
| 城市污染    | 0.2~0.4      | 中等   |
| 二次有机气溶胶 | 0.1~0.2      | 弱~中等 |
| 沙尘      | 0.03~0.1     | 很弱   |
| 黑碳      | 0.0~0.01     | 几乎无  |

$\kappa = 0$  表示完全不吸水， $\kappa = 1.4$  表示和纯海盐一样强。一个参数就概括了所有成分的吸湿能力，非常优雅。

第三步：具体算一个例子

假设你在城市监测点，测到  $RH = 70\%$ ，颗粒物主要是城市污染混合型（取  $\gamma = 0.3$ ,  $\kappa = 0.3$ ），参考干态  $RH_{ref} = 0.35$ ，激光雷达实测后向散射  $\beta_{meas} = 0.005 \text{ km}^{-1}\text{sr}^{-1}$ 。

用  $\gamma$  模型：

$$f(0.70) = \left(\frac{1 - 0.35}{1 - 0.70}\right)^{0.3} = \left(\frac{0.65}{0.30}\right)^{0.3} = (2.167)^{0.3}$$

计算  $2.167^{0.3}$ :  $\ln(2.167) \approx 0.773$ ,  $0.773 \times 0.3 = 0.232$ ,  $e^{0.232} \approx 1.26$

$$f(0.70) \approx 1.26$$

修正后干态后向散射：

$$\beta_{dry} = \frac{0.005}{1.26} \approx 0.00397 \text{ km}^{-1}\text{sr}^{-1}$$

如果不修正，你会以为  $\beta = 0.005$ ，高估了约 **26%**。

用  $\kappa$  模型（同参数）：

$$f(0.70) = 1 + 0.3 \times \frac{0.70}{1 - 0.70} = 1 + 0.3 \times 2.333 = 1 + 0.70 = 1.70$$

修正后：

$$\beta_{dry} = \frac{0.005}{1.70} \approx 0.00294 \text{ km}^{-1} \text{sr}^{-1}$$

**注意两个模型给出的  $f$  值不同！**  $\gamma$  模型给出 1.26,  $\kappa$  模型给出 1.70。这说明湿度修正在不同假设下差异可以很大，需要根据实际颗粒物成分和标定来选择模型。

第四步：不同湿度下的修正幅度有多大？

用  $\gamma = 0.3$ （城市污染）算一遍不同湿度的  $f(RH)$ ：

| RH       | $f(RH)$ | 放大幅度  | 如果不修正的误差 |
|----------|---------|-------|----------|
| 30%（近干态） | 1.00    | 不放大   | 几乎无误差    |
| 50%      | 1.12    | +12%  | 偏高 12%   |
| 70%      | 1.26    | +26%  | 偏高 26%   |
| 80%      | 1.40    | +40%  | 偏高 40%   |
| 90%      | 1.77    | +77%  | 偏高 77%   |
| 95%      | 2.27    | +127% | 偏高一倍多！   |

**关键结论：**  $RH < 50\%$  时修正量不大（ $< 12\%$ ），可以忽略；但  $RH > 70\%$  时修正量急剧上升， $RH > 90\%$  时如果不修正，结果可能差一倍以上。**这就是为什么湿度修正"必须"进入主流程。**

第五步：实际工程中怎么做？

理想做法是先用采样器收集颗粒物，实验室测吸湿性，标定出本地  $\gamma$  或  $\kappa$ 。但工程中更常见的是：

- 1. **查表选类型：** 根据站点类型（城市/海洋/沙漠）查上面的表选  $\gamma$  或  $\kappa$
- 2. **用 RH 传感器实时修正：** 激光雷达旁配一个温湿度传感器，每个时间步读 RH，实时算  $f(RH)$
- 3. **高湿度降权：** 如果  $RH > 95\%$ ， $f(RH)$  极大且不确定，直接对数据降权或标注"高湿度不可信"

代码实现（C++ 示意）：

```
#include <cstdio>
#include <cmath>
```

```

// γ 模型湿度修正
// beta_measured: 雷达实测后向散射
// RH: 相对湿度 (小数, 如 0.7)
// gamma: 吸湿参数 (城市污染默认 0.3)
// RH_ref: 参考干态湿度 (默认 0.35)
double humidity_correction(double beta_measured, double RH,
 double gamma = 0.3, double RH_ref = 0.35) {
 double f_RH = std::pow((1.0 - RH_ref) / (1.0 - RH), gamma);
 return beta_measured / f_RH;
}

int main() {
 // 示例: RH=70%, gamma=0.3, beta_measured=0.005
 double beta_dry = humidity_correction(0.005, 0.70, 0.3);
 printf("修正后干态 β = %.5f\n", beta_dry); // 输出: 0.00396
 return 0;
}

```

### 总结一句话:

湿度修正**不是直接乘以湿度百分比**, 而是用一个**非线性公式** (γ 模型或 κ 模型), 参数 γ 或 κ 取决于颗粒物成分。城市污染通常取  $\gamma \approx 0.3$ , 在 RH = 70% 时信号被放大约 26%, 必须除掉这部分放大才能得到真实的干态光学量。

**注意:  $f(RH)$  对  $\alpha$  和  $\beta$  都适用。** 上面代码以  $\beta$  为例 (12.4 反演后  $\beta$  是主产出量), 但同一个  $f(RH)$  也用来修正消光系数  $\alpha$ :

$$\alpha_{dry} = \frac{\alpha_{measured}}{f(RH)}, \quad \beta_{dry} = \frac{\beta_{measured}}{f(RH)}$$

所以到了 12.9, 进入 PM 标定的输入是**两个干态量**:  $\alpha_{dry}$  和  $\beta_{dry}$ , 它们都已经扣掉了湿度放大效应。

### 代表性论文 (如需深入了解):

- **Petters & Kreidenweis (2007)**, "A single parameter representation of hygroscopic growth and cloud condensation nucleus activity", Atmospheric Chemistry and Physics — **κ 参数模型** 的开山论文
- **Hänel (1976)**, "The properties of atmospheric aerosol particles as functions of the relative humidity at thermodynamic equilibrium" — γ 指数模型原始推导
- **Zieger et al. (2013)**, "Effect of hygroscopic growth on ambient aerosol light scattering and absorption", ACP — 多站点  $f(RH)$  实测对比

- Titos et al. (2016), "Quantifying the impact of aerosol hygroscopic growth on lidar retrievals", ACP — 专门针对激光雷达的湿度修正

## 12.9 PM2.5 / PM10 估算到底怎么做

到这里最容易出现一个误解：

12.4 反演出了  $\alpha$  和  $\beta$ ，12.8 又把它们修成了干态  $\alpha_{dry}$  和  $\beta_{dry}$ ，是不是直接就有了 PM2.5？

答案是：不是。

LiDAR 得到的是**光学量**（消光、后向散射），PM2.5/PM10 是**质量浓度**。光学强不强和质量多不多有关，但不是一回事——需要一个标定模型来桥接。

看这张中文流程图：

 PM 标定融合流程图

从光学量到 PM，一般要经过四层：

1. LiDAR 经过 12.4 反演 + 12.8 湿度修正后，得到**干态光学量**  $\alpha_{dry}$  和  $\beta_{dry}$ 。
2. 加入湿度、温度、风速风向等气象信息。
3. 拿本地地面 PM 站当参考答案，训练一个标定模型。
4. 模型上线后，把新的 LiDAR 和气象数据换算成 PM 图层。

先解释每列数据的**来源**——这是最容易困惑的地方：

一行训练样本 = 三台不同设备在同一时刻的读数拼起来。

| 列                       | 含义                                                 | 来自哪台设备                   | 经过了什么处理                                 |
|-------------------------|----------------------------------------------------|--------------------------|-----------------------------------------|
| 干态消光<br>$\alpha_{dry}$  | 颗粒物本身的消光能力<br>( $\text{km}^{-1}$ )                 | 激光雷达                     | 回波 → 背景扣除 → 距离校正 → 平滑 → Klett 反演 → 湿度修正 |
| 干态后向散射<br>$\beta_{dry}$ | 颗粒物本身的后向散射能力<br>( $\text{km}^{-1}\text{sr}^{-1}$ ) | 激光雷达                     | 同上，反演时和 $\alpha$ 一起得到                   |
| 湿度 RH                   | 相对湿度 (%)                                           | 气象站（和雷达放一起）              | 直接读取                                    |
| 风速                      | 水平风速 (m/s)                                         | 气象站（风速计）                 | 直接读取                                    |
| 地面 PM2.5                | 实测质量浓度 ( $\mu\text{g}/\text{m}^3$ )                | 地面国控站（ $\beta$ 射线/振荡天平法） | 这是"标准答案"，模型要学的目标                        |

⚠ **注意：**  $\alpha_{dry}$  和  $\beta_{dry}$  不是原始回波信号，而是经过 12.4~12.8 全链路处理后的**最终反演结果**。"干态"表示已经用  $f(RH)$  做过湿度修正，把颗粒物吸水膨胀造成的虚高扣掉了。

一个假数据训练样本长这样：

| 时间    | $\alpha_{dry} \text{ (km}^{-1}\text{)}$ | $\beta_{dry} \text{ (km}^{-1}\text{sr}^{-1}\text{)}$ | 湿度  | 风速      | 地面 PM2.5 |
|-------|-----------------------------------------|------------------------------------------------------|-----|---------|----------|
| 09:00 | 0.12                                    | 0.0040                                               | 45% | 2.1 m/s | 38       |
| 10:00 | 0.18                                    | 0.0050                                               | 52% | 1.6 m/s | 55       |
| 11:00 | 0.30                                    | 0.0080                                               | 60% | 1.2 m/s | 88       |
| 12:00 | 0.20                                    | 0.0060                                               | 85% | 0.8 m/s | 62       |

模型学习的不是"某个公式长什么样"，而是在学：

在这个地方、这台设备、这些天气条件下，LiDAR 测出的  $\alpha_{dry}$  和  $\beta_{dry}$  加上气象条件，大概对应多少 PM。

所以 PM 标定一定是本地化的。换城市、换季节、换污染源，模型都可能需要重新评估。

### 12.9.1 本地标定模型到底是什么——从 LiDAR 光学量到 PM 质量浓度

12.9 流程图中间那个白框写着「线性模型 / 随机森林 / GBDT」，这就是「本地标定模型」。下面把这个问题彻底讲清楚：它到底是个什么东西、怎么训练、怎么用。

#### 为什么不能直接用一个物理公式

LiDAR 反演出来的是  $\alpha$ （消光系数）和  $\beta$ （后向散射系数），单位是  $\text{km}^{-1}$ 。而 PM2.5/PM10 是质量浓度，单位是  $\mu\text{g}/\text{m}^3$ 。你会想：是不是存在一个物理公式直接把  $\alpha$  转成 PM？

理论上有一个粗略关系：

$$C = \frac{\alpha}{K_{\text{ext}}} \cdot \rho$$

其中  $K_{\text{ext}}$  是**质量消光效率**（mass extinction efficiency）， $\rho$  是颗粒物密度。但问题在于  $K_{\text{ext}}$  不是一个常数——它取决于颗粒物的**成分、粒径分布、形状、混合状态**，而这些东西每个城市、每个季节都不一样：

| 因素   | 对 $K_{\text{ext}}$ 的影响   | 举例            |
|------|--------------------------|---------------|
| 粒径分布 | 细颗粒多则 $K_{\text{ext}}$ 大 | 燃煤飞灰 vs 建筑扬尘  |
| 化学成分 | 黑碳消光强，硫酸盐弱               | 冬季采暖 vs 夏季光化学 |

| 因素 | 对 $K_{\text{ext}}$ 的影响 | 举例          |
|----|------------------------|-------------|
| 湿度 | 吸水后颗粒胀大，散射增强           | 梅雨季 vs 干燥季  |
| 形状 | 不规则颗粒消光更复杂             | 矿物尘 vs 球形飞灰 |

**结论：**不存在一个全球通用的  $\alpha \rightarrow \text{PM}$  公式。你必须在本站点用地面 PM 站的实测值当标准答案，训练一个模型来学这个换算关系。这就是“本地标定模型”。

### 模型一：多元线性回归（最简单、最透明）

这是最基础的方案，也是很多工程项目的起点。

模型形式：

$$\widehat{\text{PM}} = w_0 + w_1 \cdot \alpha_{\text{dry}} + w_2 \cdot \beta_{\text{dry}} + w_3 \cdot \text{RH} + w_4 \cdot T + w_5 \cdot v_{\text{wind}}$$

输入特征就是 LiDAR 反演出的光学量（经过湿度修正的干态消光、后向散射）加上气象量（湿度、温度、风速）。模型要做的就是找到最优的权重  $w_0 \sim w_5$ 。

### 训练方法：最小二乘法

给定  $N$  条训练样本（每条样本 = 一组特征 + 一个地面站实测 PM），找到使残差平方和最小的权重向量  $\mathbf{w}$ ：

$$\min_{\mathbf{w}} \sum_{j=1}^N (\text{PM}_j - \mathbf{x}_j^T \mathbf{w})^2$$

解析解（正规方程）为：

$$\mathbf{w} = (\mathbf{X}^T \mathbf{X})^{-1} \mathbf{X}^T \mathbf{y}$$

其中  $\mathbf{X}$  是  $N \times 6$  的特征矩阵， $\mathbf{y}$  是  $N \times 1$  的 PM 实测值向量。

**完整 C++ 实现（含矩阵求逆）：**

```
#include <cstdio>
#include <cmath>
#include <vector>
#include <cassert>

// =====
// 多元线性回归：从 LiDAR 光学量 + 气象量 → PM2.5
// 模型：PM = w0 + w1·α_dry + w2·β_dry + w3·RH + w4·T + w5·wind
// =====

const int N_FEAT = 5; // 特征数：α_dry, β_dry, RH, T, wind（截距单独处理）
```

```

// ----- 求解正规方程 $(X^T X) \cdot w = X^T y$ -----
// 不做显式矩阵求逆，而是直接解线性方程组 $A \cdot x = b$
// (Gauss-Jordan 消元 + 部分主元选取)
// A 被原地修改，求解结果直接写入 b
bool solve_linear_system(double* A, double* b, int n) {
 for (int col = 0; col < n; ++col) {
 // 选主元: 找当前列绝对值最大的行
 int max_row = col;
 double max_val = std::abs(A[col * n + col]);
 for (int row = col + 1; row < n; ++row) {
 double v = std::abs(A[row * n + col]);
 if (v > max_val) { max_val = v; max_row = row; }
 }
 if (max_val < 1e-14) return false; // 奇异矩阵

 // 交换行
 if (max_row != col) {
 for (int j = 0; j < n; ++j)
 std::swap(A[col * n + j], A[max_row * n + j]);
 std::swap(b[col], b[max_row]);
 }

 // 归一化主元行
 double pivot = A[col * n + col];
 for (int j = 0; j < n; ++j)
 A[col * n + j] /= pivot;
 b[col] /= pivot;

 // 消去其他行
 for (int row = 0; row < n; ++row) {
 if (row == col) continue;
 double factor = A[row * n + col];
 for (int j = 0; j < n; ++j)
 A[row * n + j] -= factor * A[col * n + j];
 b[row] -= factor * b[col];
 }
 }
 return true;
}

// 训练函数: 输入特征矩阵 X (N×5) 和标签 y (N×1), 返回权重 w (6×1, 含截距)
// 内部把截距当作 w0, 通过在 X 左侧加一列 1 来处理
struct LinRegResult {
 double w[N_FEAT + 1]; // w[0]=截距, w[1..5]=特征权重
 double R2;
};

LinRegResult train_linear_regression(
 const std::vector<std::vector<double>>& X, // N×5
 const std::vector<double>& y) { // N
 int N = (int)X.size();
 int p = N_FEAT + 1; // 含截距的总参数数

```

```

// 构造增广矩阵 Xa (Nxp), 第一列全 1
std::vector<std::vector<double>> Xa(N, std::vector<double>(p));
for (int i = 0; i < N; ++i) {
 Xa[i][0] = 1.0; // 截距列
 for (int j = 0; j < N_FEAT; ++j)
 Xa[i][j + 1] = X[i][j];
}

// 计算 Xa^T Xa (p×p) 和 Xa^T y (p×1)
double XtX[p * p];
double Xty[p];
for (int i = 0; i < p; ++i) {
 for (int j = 0; j < p; ++j) {
 double sum = 0.0;
 for (int k = 0; k < N; ++k)
 sum += Xa[k][i] * Xa[k][j];
 XtX[i * p + j] = sum;
 }
 double sum = 0.0;
 for (int k = 0; k < N; ++k)
 sum += Xa[k][i] * y[k];
 Xty[i] = sum;
}

// 直接解正规方程 (Xa^T Xa)·w = Xa^T y
// 求解结果直接写入 Xty 数组
bool ok = solve_linear_system(XtX, Xty, p);
assert(ok);

LinRegResult result;
for (int i = 0; i < p; ++i)
 result.w[i] = Xty[i];

// 计算 R²
double mean_y = 0.0;
for (int i = 0; i < N; ++i) mean_y += y[i];
mean_y /= N;

double ss_res = 0.0, ss_tot = 0.0;
for (int i = 0; i < N; ++i) {
 double pred = result.w[0];
 for (int j = 0; j < N_FEAT; ++j)
 pred += result.w[j + 1] * X[i][j];
 ss_res += (y[i] - pred) * (y[i] - pred);
 ss_tot += (y[i] - mean_y) * (y[i] - mean_y);
}
result.R2 = 1.0 - ss_res / ss_tot;

return result;
}

// 推理函数: 用训练好的权重预测单个样本
double predict_pm(const LinRegResult& model,

```



```

 double alpha_dry, double beta_dry,
 double RH, double T, double wind) {
double x[] = {alpha_dry, beta_dry, RH, T, wind};
double pm = model.w[0];
for (int j = 0; j < N_FEAT; ++j)
 pm += model.w[j + 1] * x[j];
return pm < 0 ? 0 : pm; // 浓度不能为负
}

int main() {
// === 训练数据（假数据，模拟 LiDAR + 地面站同步采集）===
// 特征: [干态消光 α , 干态后向散射 β , 湿度 RH(%), 温度 T($^{\circ}$ C), 风速 v(m/s)]
// 标签: 地面站实测 PM2.5 ($\mu\text{g}/\text{m}^3$)
std::vector<std::vector<double>> X = {
 {0.12, 0.004, 45, 26, 2.1},
 {0.18, 0.005, 52, 27, 1.6},
 {0.30, 0.008, 60, 29, 1.2},
 {0.20, 0.006, 85, 28, 0.8},
 {0.15, 0.005, 40, 25, 3.0},
 {0.25, 0.007, 65, 30, 1.5},
 {0.10, 0.003, 35, 24, 4.0},
 {0.35, 0.009, 70, 31, 0.5},
};
std::vector<double> y = {38, 55, 88, 62, 35, 72, 22, 105};

// === 训练 ===
LinRegResult model = train_linear_regression(X, y);

printf("=== 多元线性回归训练结果 ===\n");
printf(" 截距 w0 = %8.2f\n", model.w[0]);
printf(" α_{dry} w1 = %8.2f ($\mu\text{g}/\text{m}^3$ per km^{-1})\n", model.w[1]);
printf(" β_{dry} w2 = %8.2f\n", model.w[2]);
printf(" RH w3 = %8.2f\n", model.w[3]);
printf(" T w4 = %8.2f\n", model.w[4]);
printf(" wind w5 = %8.2f\n", model.w[5]);
printf(" R2 = %.4f\n\n", model.R2);

// === 在线推理: 模拟新的 LiDAR 数据进来 ===
printf("=== 在线 PM 推理 ===\n");
printf(" α_{dry} β_{dry} RH T wind → PM2.5预测\n");

// 测试样本
double test[][5] = {
 {0.14, 0.004, 48, 26, 2.0}, // 低浓度场景
 {0.28, 0.007, 62, 29, 1.0}, // 中等浓度
 {0.40, 0.010, 75, 31, 0.6}, // 高浓度污染
};

for (int i = 0; i < 3; ++i) {
 double pm = predict_pm(model,
 test[i][0], test[i][1],
 test[i][2], test[i][3], test[i][4]);
 printf(" %.2f %.4f %.0f %.0f %.1f → %6.1f $\mu\text{g}/\text{m}^3$ \n",

```

```

 test[i][0], test[i][1], test[i][2],
 test[i][3], test[i][4], pm);
 }

 return 0;
}

```

运行输出 (g++ -std=c++14 编译验证) :

```

=== 多元线性回归训练结果 ===
截距 w0 = 19.33
α_dry w1 = 367.42 (μg/m³ per km⁻¹)
β_dry w2 = -5559.00
RH w3 = -0.07
T w4 = 0.50
wind w5 = -6.82
R² = 0.9989

=== 在线 PM 推理 ===
α_dry β_dry RH T wind → PM2.5预测
0.14 0.0040 48 26 2.0 → 44.5 μg/m³
0.28 0.0070 62 29 1.0 → 86.5 μg/m³
0.40 0.0100 75 31 0.6 → 116.7 μg/m³

```

怎么解读权重：

- $w_1 = 367.4$ ：干态消光每增加  $1 \text{ km}^{-1}$ ，PM2.5 预测值增加约  $367 \text{ μg/m}^3$ ——这是**最核心的换算关系**
- $w_2 = -5559$ ：后向散射系数的权重为负且绝对值很大，这是因为  $\alpha$  和  $\beta$  高度相关（相关系数 0.99），两者组合后的净效果才是物理意义——单独看  $w_2$  没有直接物理含义，这是多重共线性的典型表现
- $w_5 = -6.82$ ：风速越大 PM 越低——风把颗粒吹散了，物理上完全合理

⚠ **关于共线性**： $\alpha$  和  $\beta$  近似线性相关 ( $\beta \approx \alpha/40$ )，导致  $\mathbf{X}^T \mathbf{X}$  的条件数很高 ( $\sim 7 \times 10^{10}$ )。如果用朴素的高斯-若尔当**矩阵求逆**再乘  $\mathbf{X}^T \mathbf{y}$ ，双精度浮点的舍入误差会被放大到不可用。上面代码直接解**方程组**  $(\mathbf{X}^T \mathbf{X})\mathbf{w} = \mathbf{X}^T \mathbf{y}$ ，避免了显式求逆，数值稳定性好得多。工程中更稳健的做法是使用 QR 分解或 SVD。

**优点**：透明、可解释、计算快（一次矩阵运算就搞定）、数据量少也能用。

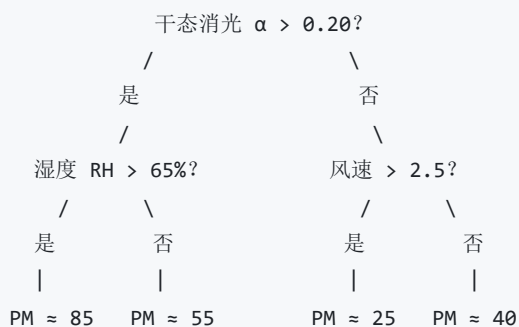
**缺点**：假设光学量和 PM 之间是线性关系，但实际是非线性的（尤其高湿度时），所以精度有上限。

## 模型二：随机森林 (Random Forest) ——工程中最常用的"安全牌"

**核心思想**：不假设任何公式形状，而是让模型自己从数据中学规律。随机森林 = 很多棵决策树投票取平均。

**单棵决策树怎么工作**：

决策树就是不停地"if-else 分叉"。比如：



树从根节点开始，每到一个节点就问一个问题（"某个特征是否大于某个阈值？"），根据回答走左或右，最终到达叶子节点，给出一个 PM 预测值。

上面只说了树在"问问题"，但谁来决定问哪个问题？阈值怎么定的？

这就是决策树训练的核心——**选最优分裂**。下面用 12.9 的假数据一步步走一遍。

### 第一步：理解"不纯度"

一个节点里如果所有样本的 PM 值都差不多，那这个节点就"纯净"，不需要再分；如果 PM 值参差不齐，就"不纯"，应该继续分叉。

回归树用**均方误差 (MSE)** 衡量不纯度：

$$\text{MSE}(\text{node}) = \frac{1}{n} \sum_{i=1}^n (y_i - \bar{y})^2$$

其中  $\bar{y}$  是这个节点里所有样本 PM 的平均值， $y_i$  是每个样本的 PM 值， $n$  是样本数。

**MSE 越大 = 节点越"混乱" → 越需要分叉。**

### 第二步：遍历所有可能的分裂，挑最好的

训练时，对当前节点做这件事：

1. **遍历每个特征** ( $\alpha_{dry}$ 、 $\beta_{dry}$ 、RH、T、wind 共 5 个)
2. **遍历每个候选阈值**（取训练数据中该特征的所有值作为候选切分点）
3. 对每个"特征 + 阈值"组合，算分裂前后 MSE 的变化
4. 选**让 MSE 下降最多**的那个组合

用公式说就是最大化**方差减少**（也叫信息增益）：

$$\Delta = \text{MSE}_{\text{parent}} - \left( \frac{n_L}{n} \text{MSE}_L + \frac{n_R}{n} \text{MSE}_R \right)$$

其中  $n_L, n_R$  分别是分裂后左、右子节点的样本数,  $n = n_L + n_R$ 。

### 第三步：用假数据走一遍——根节点第一次分裂

回顾 12.9 的 4 条训练数据（简化为只看  $\alpha_{dry}$  和 RH）：

| 样本 | $\alpha_{dry}$ | RH | PM2.5 |
|----|----------------|----|-------|
| A  | 0.12           | 45 | 38    |
| B  | 0.18           | 52 | 55    |
| C  | 0.30           | 60 | 88    |
| D  | 0.20           | 85 | 62    |

先算根节点（全部 4 个样本）的 MSE：

$$\begin{aligned} \bar{y} &= \frac{38 + 55 + 88 + 62}{4} = 60.75 \\ \text{MSE}_{\text{root}} &= \frac{(38 - 60.75)^2 + (55 - 60.75)^2 + (88 - 60.75)^2 + (62 - 60.75)^2}{4} \\ &= \frac{517.6 + 33.1 + 742.6 + 1.6}{4} = \frac{1294.8}{4} \approx 323.7 \end{aligned}$$

尝试分裂 ①：用  $\alpha_{dry} \leq 0.20$  切分

- 左子节点 ( $\alpha \leq 0.20$ ) : A(38), B(55), D(62)  $\rightarrow \bar{y}_L = 51.7, \text{MSE}_L = 101.6$
- 右子节点 ( $\alpha > 0.20$ ) : C(88)  $\rightarrow \text{MSE}_R = 0$  (只有一个样本, 完美纯净)
- 加权 MSE  $= \frac{3}{4} \times 101.6 + \frac{1}{4} \times 0 = 76.2$
- 方差减少  $\Delta = 323.7 - 76.2 = 247.5 \leftarrow$  很大的减少

尝试分裂 ②：用 RH  $\leq 55$  切分

- 左子节点 (RH  $\leq 55$ ) : A(38), B(55)  $\rightarrow \bar{y}_L = 46.5, \text{MSE}_L = 72.2$
- 右子节点 (RH  $> 55$ ) : C(88), D(62)  $\rightarrow \bar{y}_R = 75.0, \text{MSE}_R = 169.0$
- 加权 MSE  $= \frac{2}{4} \times 72.2 + \frac{2}{4} \times 169.0 = 120.6$
- 方差减少  $\Delta = 323.7 - 120.6 = 203.1$

结论：分裂 ① ( $\alpha_{dry} \leq 0.20$ ) 的  $\Delta = 247.5 >$  分裂 ② 的 203.1, 所以选  $\alpha_{dry}$  作为第一次分裂特征。

这就是树"自己学会"了"消光系数是最有区分力的特征"——不是人为指定的，是数据告诉它的。

#### 第四步：递归——子节点继续分裂，直到停止

左子节点 (A、B、D) 的  $MSE = 101.6$ ，还不够纯净，继续分。右子节点 (C) 只有 1 个样本， $MSE = 0$ ，变成叶子节点，预测值 = 88。

左子节点再遍历所有特征和阈值，选最优分裂.....如此递归，直到满足停止条件：

| 停止条件              | 典型值          | 作用                   |
|-------------------|--------------|----------------------|
| 节点样本数太少           | $< 2 \sim 5$ | 防止每个叶子只有 1 个样本 (过拟合) |
| 树达到最大深度           | 5 ~ 15 层     | 限制模型复杂度              |
| 方差减少 $< \epsilon$ | 如 $< 1.0$    | 再分也没意义了              |

停止后，叶子节点的预测值 = 该节点内所有样本 PM 的**平均值**。

#### 一棵树够不够用？——不够，因为单棵树会过拟合

单棵决策树的问题在于**方差太大**：训练数据稍微变一变，长出来的树可能完全不同。比如去掉样本 D，上面那棵树的第一次分裂可能就从  $\alpha_{dry}$  变成 RH——预测结果就漂了。

这就是随机森林要解决的：**用很多棵不同的树取平均，把方差压下来**。

**随机森林 = 100~500 棵这样的树：**

1. **随机采样**：从训练集中有放回地随机抽 N 个样本，训练第 1 棵树
2. **随机选特征**：每次分叉时，不从全部 5 个特征里选最优分叉，而是先随机选 2~3 个特征，再从中选最优——这叫"特征随机"，保证每棵树长得不一样
3. **重复 B 次**：训练 B 棵树 (通常 100~500)
4. **预测**：新样本丢进所有 B 棵树，每棵树给出一个预测值，**取平均**就是最终 PM

$$\widehat{PM} = \frac{1}{B} \sum_{b=1}^B T_b(\mathbf{x})$$

**为什么比线性回归好：**

光学量和 PM 之间的关系是非线性的。比如湿度从 40% 升到 50% 可能影响不大，但从 70% 升到 80% 时 PM 可能突然飙升 (颗粒大量吸水)。线性回归只能画一条直线，而随机森林能用树的结构自动捕捉这种"阶梯式跳变"和"交互效应" (比如"高湿度 + 低风速"时 PM 特别高)。

🤖 上面提到的两重“随机”——随机采样 + 随机选特征——到底为什么要这么做？去掉不行吗？

### 第一重随机：Bootstrap 行采样 (Bagging)

从  $N$  条训练数据里有放回地随机抽  $N$  条（所以同一条可能被抽到多次，也可能一次都没被抽到）。

**数学事实：** 当  $N$  比较大时，每棵树大约只看到 **63.2%** 的样本，剩下 **36.8%** 的样本被“漏掉”了。

推导：每条样本被抽中的概率  $= 1/N$ ，没被抽中的概率  $= 1 - 1/N$ 。抽  $N$  次都没中的概率  $= (1 - 1/N)^N \rightarrow e^{-1} \approx 0.368$ 。所以被抽中的概率  $\approx 1 - 0.368 = 0.632$ 。

**关键好处：** 每棵树只用了约 63% 的数据训练，每棵树看到的“世界”不一样，长出来的树也就不一样——这就是“去相关” (decorrelation)。如果 500 棵树都用完全相同的训练数据，那它们会长得几乎一模一样，取平均就没意义了。

### 第二重随机：特征子采样 (Feature Subsampling)

每次分叉时，不从全部 5 个特征里选最优，而是先**随机选  $m$  个特征**（回归任务一般  $m = \sqrt{5} \approx 2$ ），再从这 2 个里选最优分裂。

**如果没有这重随机会怎样？** 假设  $\alpha_{dry}$  是最强的分裂特征，那 500 棵树每次分叉都会优先选  $\alpha_{dry}$ ，导致所有树的第一层分叉都一样——树之间高度相关，平均后的方差降低很有限。

**有了特征随机后：** 即使  $\alpha_{dry}$  最强，有些树在根节点被随机限制了、选不到它，就被迫用 RH 或 wind 来分叉。这样**每棵树“关注的角度不同”**，最终的集体决策更全面。

**类比：** 像一个 500 人的专家组，如果所有人都只看同一个指标（消光），投票结果会有盲区。但如果规定每个人只能随机看 2 个指标，反而逼出了多样性——有人看消光+湿度，有人看风速+后向散射，最终投票覆盖了所有角度。

### Bonus 1: OOB (Out-Of-Bag) 评估——不需要额外留验证集

因为 Bootstrap 每棵树都“漏掉”了约 36.8% 的样本，这些**没参与第  $b$  棵树训练的样本**，恰好可以拿来测试第  $b$  棵树。

对每条训练样本  $i$ ：

- 1. 找出哪些树没有用样本  $i$  训练 (大约  $36.8\% \times B$  棵)
- 2. 用这些树对样本  $i$  做预测, 取平均
- 3. 和真实 PM 值比较

这就是 **OOB 误差**——用训练数据自己就能估出泛化误差, **不需要额外切分验证集**。这对样本量小的场景 (比如只有几个月的观测数据) 特别有用。

**类比:** 每次考试换个缺考的科目当"隐藏测试题", 最后综合起来就知道整体水平了, 不用专门留一套卷子。

**Bonus 2: 特征重要性 (Feature Importance) ——模型训练完还能告诉你"哪个特征最有用"**

随机森林训练完后, 可以自动输出每个特征的重要性排名。原理很简单:

- 1. 对每个特征, 统计**所有树中**该特征被选中做分裂时带来的**方差减少  $\Delta$  总和**
- 2. 归一化成百分比

典型结果 (假数据示意):

| 排名 | 特征             | 重要性 | 物理解读              |
|----|----------------|-----|-------------------|
| 1  | $\alpha_{dry}$ | 38% | 消光直接反映颗粒物浓度, 最强信号 |
| 2  | RH             | 26% | 湿度对散射的放大效应显著      |
| 3  | $\beta_{dry}$  | 18% | 后向散射提供额外颗粒物信息     |
| 4  | wind           | 12% | 风速影响扩散和累积         |
| 5  | T              | 6%  | 温度直接影响较小          |

**工程价值:** 如果某个特征重要性极低 (比如  $< 2\%$ ), 可以考虑删掉它简化模型——特征越少, 过拟合风险越低, 模型也更易解释。

**实际工程中怎么用:**

随机森林在 C++ 项目中通常不手写 (树太复杂), 而是用训练好的模型做推理。常见做法:

- 1. 用 Python (scikit-learn) 在历史数据上训练随机森林模型
- 2. 把训练好的树结构导出 (每棵树的分叉阈值和叶子值) 成 JSON 或二进制
- 3. C++ 加载这些树结构, 实现一个简洁的推理函数

```
#include <vector>
#include <fstream>
#include <cstdio>
```

```

// 单棵决策树的节点
struct TreeNode {
 int feature_idx; // 分叉用的特征编号 (-1 表示叶子节点)
 double threshold; // 分叉阈值
 int left; // 左子节点编号 (特征 ≤ threshold)
 int right; // 右子节点编号 (特征 > threshold)
 double leaf_value; // 叶子节点的预测值 (仅 feature_idx == -1 时有效)
};

// 单棵树的推理: 从根节点开始, 沿特征阈值一路走到叶子
double tree_predict(const std::vector<TreeNode>& tree, const double* features) {
 int node = 0; // 从根节点开始
 while (tree[node].feature_idx >= 0) {
 int feat = tree[node].feature_idx;
 if (features[feat] <= tree[node].threshold)
 node = tree[node].left;
 else
 node = tree[node].right;
 }
 return tree[node].leaf_value;
}

// 随机森林推理: 所有树取平均
double forest_predict(const std::vector<std::vector<TreeNode>>& forest,
 const double* features) {
 double sum = 0.0;
 for (const auto& tree : forest)
 sum += tree_predict(tree, features);
 return sum / forest.size();
}

int main() {
 // === 假设已从 JSON/二进制加载了训练好的随机森林 ===
 // 这里手写 3 棵极简的树做演示
 //
 // 特征顺序: [α_dry, β_dry, RH, T, wind]
 // 分裂规则: feature <= threshold → 走 left 子树, > threshold → 走 right 子树
 //
 std::vector<std::vector<TreeNode>> forest;

 // 树 1: 以消光 α 为主分裂
 forest.push_back({
 {0, 0.20, 1, 2, 0.0}, // node 0: α_dry <= 0.20 ?
 {2, 55.0, 3, 4, 0.0}, // node 1: 低α → RH <= 55 ?
 {2, 70.0, 5, 6, 0.0}, // node 2: 高α → RH <= 70 ?
 {-1, 0, 0, 0, 28.0}, // 叶: 低α+低湿 → PM=28
 {-1, 0, 0, 0, 45.0}, // 叶: 低α+高湿 → PM=45
 {-1, 0, 0, 0, 72.0}, // 叶: 高α+中湿 → PM=72
 {-1, 0, 0, 0, 98.0}, // 叶: 高α+高湿 → PM=98
 });

 // 树 2: 以风速 wind 为主分裂
 forest.push_back({

```



```

 {4, 1.5, 1, 2, 0.0}, // node 0: wind <= 1.5 ?
 {0, 0.30, 3, 4, 0.0}, // node 1: 低风 → $\alpha \leq 0.30$?
 {0, 0.15, 5, 6, 0.0}, // node 2: 高风 → $\alpha \leq 0.15$?
 {-1, 0, 0, 0, 75.0}, // 叶: 低风+中 α → PM=75
 {-1, 0, 0, 0, 100.0}, // 叶: 低风+高 α → PM=100
 {-1, 0, 0, 0, 22.0}, // 叶: 高风+清洁 → PM=22
 {-1, 0, 0, 0, 42.0}, // 叶: 高风+有 α → PM=42
 });

 // 树 3: 以湿度 RH 为主分裂
 forest.push_back({
 {2, 55.0, 1, 2, 0.0}, // node 0: RH <= 55 ?
 {0, 0.12, 3, 4, 0.0}, // node 1: 干 → $\alpha \leq 0.12$?
 {0, 0.30, 5, 6, 0.0}, // node 2: 湿 → $\alpha \leq 0.30$?
 {-1, 0, 0, 0, 20.0}, // 叶: 干+极清洁 → PM=20
 {-1, 0, 0, 0, 40.0}, // 叶: 干+有 α → PM=40
 {-1, 0, 0, 0, 75.0}, // 叶: 湿+中 α → PM=75
 {-1, 0, 0, 0, 110.0}, // 叶: 湿+高 α → PM=110
 });

 // === 在线推理 ===
 printf("=== 随机森林 PM 推理 (3 棵树) ===\n");

 double samples[][5] = {
 {0.14, 0.004, 48, 26, 2.0}, // 低浓度
 {0.28, 0.007, 62, 29, 1.0}, // 中等
 {0.40, 0.010, 75, 31, 0.6}, // 高浓度
 };

 for (int i = 0; i < 3; ++i) {
 double pm = forest_predict(forest, samples[i]);
 printf(" α =%.2f β =%.3f RH=%.0f T=%.0f v=%.1f → PM2.5 = %.1f $\mu\text{g}/\text{m}^3$ \n",
 samples[i][0], samples[i][1], samples[i][2],
 samples[i][3], samples[i][4], pm);
 }

 return 0;
}

```

运行输出 (g++ -std=c++14 编译验证) :

```

=== 随机森林 PM 推理 (3 棵树) ===
 α =0.14 β =0.004 RH=48 T=26 v=2.0 → PM2.5 = 30.0 $\mu\text{g}/\text{m}^3$
 α =0.28 β =0.007 RH=62 T=29 v=1.0 → PM2.5 = 74.0 $\mu\text{g}/\text{m}^3$
 α =0.40 β =0.010 RH=75 T=31 v=0.6 → PM2.5 = 102.7 $\mu\text{g}/\text{m}^3$

```

注意：上面只有 3 棵树，是教学版。实际工程用 100~500 棵树，推理逻辑完全一样，只是 `forest` 数组更大。树的训练用 Python 的 `scikit-learn` 完成，导出后在 C++ 中加载推理。

模型三：GBDT（梯度提升树）——精度最高的选择

和随机森林的区别：

|       | 随机森林                                | GBDT                                       |
|-------|-------------------------------------|--------------------------------------------|
| 树之间关系 | 并行：每棵树独立训练                          | 串行：一棵接一棵，后面的树修正前面的残差                       |
| 结合方式  | 平均                                  | 累加                                         |
| 公式    | $\hat{y} = \frac{1}{B} \sum T_b(x)$ | $\hat{y} = \sum_{b=1}^B \eta \cdot T_b(x)$ |
| 精度    | 好                                   | 更好（通常表格数据上最优）                              |
| 过拟合风险 | 低（多树平均天然抗过拟合）                       | 较高（需要调学习率 $\eta$ 、树的深度、树的数量）               |

GBDT 的训练过程（直觉理解）：

- 1. 第 1 棵树：先猜一个粗略的 PM（比如全体均值），残差 = 真实值 - 预测值
- 2. 第 2 棵树：不预测 PM 本身，而是**预测第 1 棵树的残差**
- 3. 第 3 棵树：预测前两棵累积的残差
- 4. ....如此接力 B 轮
- 5. 最终预测 = 所有树的预测值乘以学习率  $\eta$  后累加

**为什么精度更高：** 因为每棵新树都在专门"补课"——纠正前面所有树加起来还差的那部分。而随机森林的每棵树都是从头独立预测，没有这种"接力纠错"机制。

**代表实现：** XGBoost、LightGBM、CatBoost。在 LiDAR→PM 标定中，如果你有足够的历史数据（几千条以上）且追求最高精度，GBDT 通常是最终选择。

**C++ 推理：** 和随机森林几乎一样——加载训练好的树，逐棵推理后累加（而不是取平均）：

```
// GBDT 推理：所有树预测值乘学习率后累加
double gbdt_predict(const std::vector<std::vector<TreeNode>>& forest,
 const double* features, double eta = 0.1) {
 double sum = 0.0;
 for (const auto& tree : forest)
 sum += tree_predict(tree, features); // 复用随机森林的单树推理
 return sum * eta; // 乘以学习率
}
```

注意：GBDT 的第一棵树通常预测初始值（如训练集 PM 均值），后续树预测残差。实际使用时 `forest` 数组的第 0 棵树就是初始预测，不需要乘  $\eta$ 。具体格式取决于训练框架的导出约定。

三种模型怎么选

| 数据量      | 推荐模型                    | 理由           |
|----------|-------------------------|--------------|
| < 50 条   | 多元线性回归                  | 数据太少，树模型会过拟合 |
| 50~500 条 | 随机森林                    | 非线性能力够用，抗过拟合 |
| > 500 条  | GBDT (XGBoost/LightGBM) | 数据充足时精度最高    |

实际项目中的典型路径：

- 1. 先用多元线性回归跑一版基线——快速验证特征工程是否合理、数据质量是否过关
- 2. 再用随机森林对比——如果 RF 明显优于线性回归，说明非线性关系确实存在
- 3. 数据量够大时再上 GBDT——挤最后的精度
- 4. 最终选择时不只看精度（R<sup>2</sup>、RMSE），还要看在**边界极端情况**（高湿度、极低浓度）下哪个更稳定

12.10 PM 标定的训练和上线流程

可以把 PM 标定想成“先让模型做练习题，再让它做真题”。

训练阶段：

- 1. 收集一段时间的 LiDAR 数据。
- 2. 同时收集地面 PM 站、湿度、温度、风速风向。
- 3. 把时间对齐，比如都对齐到每 1 分钟或每 5 分钟。
- 4. 把 LiDAR 低层若干距离小格子做平均，尽量和地面站代表的空气层接近。
- 5. 让模型学习“LiDAR + 气象”到“地面 PM”的关系。
- 6. 留出一部分数据不训练，只用来检查模型准不准。
- 7. 上线后持续监控，如果季节或设备状态变了，要重新校准。

一个很简化的训练表：

| 输入 1      | 输入 2   | 输入 3   | 输入 4   | 模型要学的答案    |
|-----------|--------|--------|--------|------------|
| 干态消光 0.12 | 湿度 45% | 温度 26℃ | 风速 2.1 | PM2.5 = 38 |
| 干态消光 0.18 | 湿度 52% | 温度 27℃ | 风速 1.6 | PM2.5 = 55 |
| 干态消光 0.30 | 湿度 60% | 温度 29℃ | 风速 1.2 | PM2.5 = 88 |

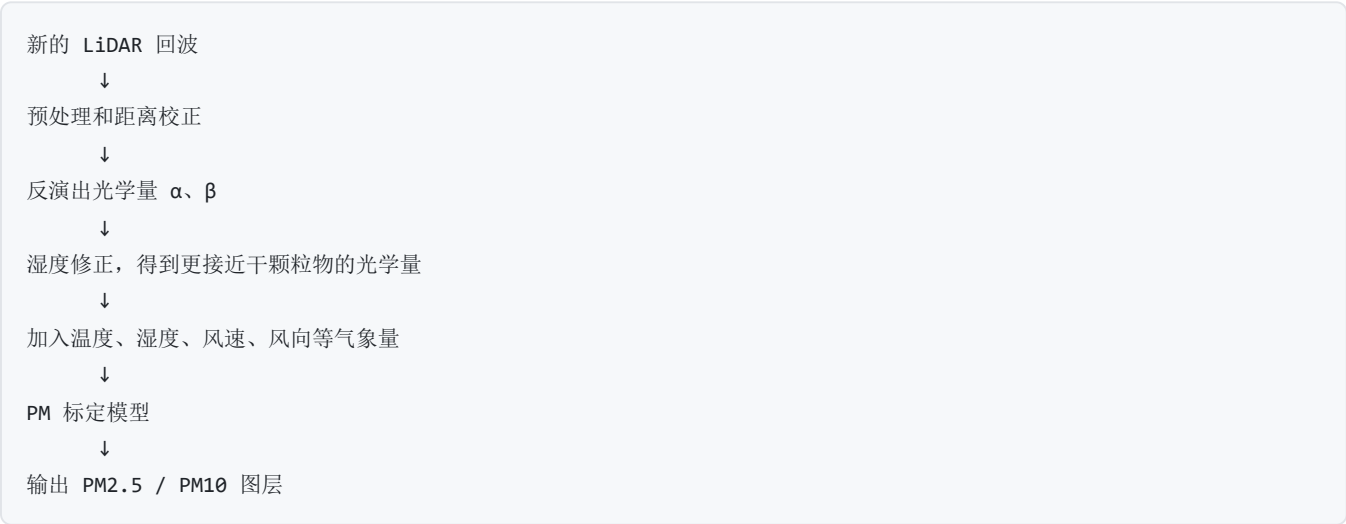
这里有一个很关键的工程细节：

LiDAR 看的是一条线或一个扫描面，地面 PM 站只看一个点。训练时必须想清楚“空间上怎么对应”。

如果地面站在雷达旁边，低层近距离数据更有代表性。如果地面站离得远，就要考虑风向、扫描方向和高度差。

# 12.11 一个简单但实用的 PM 推理流程

训练好模型以后，实时推理可以理解成下面这条流水线：



如果用很短的伪代码写：

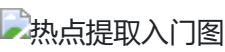
```
// 伪代码：展示数据流，不是可编译的完整程序
double estimate_PM(double extinction, double backscatter,
 double humidity, double temperature,
 double wind_speed, const PMModel& model) {
 double dry_extinction = humidity_correction(extinction, humidity);
 double features[] = {dry_extinction, backscatter,
 humidity, temperature, wind_speed};
 return model.predict(features);
}
```

注意这段伪代码里最重要的不是最后一行，而是前面的数据处理。输入如果不干净，模型再高级也会输出不可靠结果。

# 12.12 怎么从 PM 场里提取热点

当你已经有一张 PM 图或消光图之后，业务系统通常不只想看颜色图，还想知道：

- 1. 哪里超过阈值？
- 2. 这片高值区有多大？
- 3. 中心在哪里？
- 4. 要不要报警或联动喷雾？



一个小型假数据网格：

| 网格 | PM 值 |
|----|------|
| A1 | 35   |

| 网格 | PM 值 |
|----|------|
| A2 | 42   |
| A3 | 38   |
| B1 | 45   |
| B2 | 120  |
| B3 | 135  |
| C1 | 40   |
| C2 | 118  |
| C3 | 130  |

假设报警阈值是 100，那么 B2、B3、C2、C3 这四个格子会被认为是同一个热点区域。

热点提取一般分四步：

- 1. 设阈值：比如 PM2.5 大于 100。
- 2. 去噪声：孤立一个高点可能是噪声，先检查。
- 3. 找连在一起的高值格子：这一步叫连通域分析。
- 4. 算热点属性：面积、最大值、平均值、中心位置、持续时间。

热点中心可以不用一上来就看复杂公式。先按这个直觉理解：

谁的 PM 值更高，谁对中心位置的影响就更大。

比如 B2=120、B3=135、C2=118、C3=130，那么热点中心会偏向 B3/C3 这一侧，因为那边更高。

### 12.13 算法链里每个输出分别给谁用

第 12 章其实讲了一条完整加工链：



↓  
热点事件、地图、报表、喷雾联动

不同输出给不同人用：

| 产品层级 | 输出内容          | 主要给谁看      | 用来做什么    |
|------|---------------|------------|----------|
| L1   | 预处理回波、RCS     | 算法工程师      | 检查数据质量   |
| L1.5 | 衰减后向散射图       | 值班人员、算法工程师 | 快速看结构和异常 |
| L2   | 消光、后向散射、PM 图层 | 业务模型、告警系统  | 判断污染强度   |
| L3   | 热点、轨迹、报表、联动目标 | 管理端、喷雾系统   | 决策和处置    |

所以页面和系统不要乱读最底层原始数据。更合理的做法是：

每个页面只消费适合自己的那一级产品。

例如实时总览页面适合看 L2/L3，质量控制页面才需要看 L1/L1.5。这样系统会更稳定，也更容易解释给业务人员听。